

Chapter 36. Zvmm Family of Integrated Matrix Extensions

36.1. Introduction

High-performance computing and machine learning workloads depend critically on general matrix multiplication (GEMM) over a wide range of data types and precisions. Dedicated matrix-multiply accelerators often require new register state—separate matrix register files—to achieve competitive throughput, introducing substantial architectural complexity and binary interface disruption.

The Zvmm family of Integrated Matrix extensions (Zvmmm, Zvvfmm, Zvmtls, Zvmttls) takes a different approach: it accelerates matrix multiplication using *only* the $32 \times \text{VLEN}$ architected vector registers already defined by the RISC-V "V" vector extension. By interpreting groups of existing vector registers as two-dimensional matrix tiles, the Zvmm family of Integrated Matrix extensions delivers high arithmetic density without introducing any new architected state. We focus, in particular, on the computation of $C \leftarrow A \times B^T + C$, where A ($\sigma \times \lambda$), B ($\sigma \times \lambda$) and C ($\sigma \times \sigma$) are row-major matrix panels.

The extensions are designed to support implementations spanning a wide range of microarchitectures and performance points: from small, embedded in-order cores targeting low-power and area-constrained applications, to large, high-performance out-of-order implementations targeting HPC and AI workloads. A key design goal is that the same binary executes correctly—and achieves near-peak arithmetic throughput—across this entire range without recompilation.

36.2. Overview

The Zvmm family of Integrated Matrix extensions (Zvmmm, Zvvfmm, Zvmtls, Zvmttls) provides the following subextensions:

- [Matrix-multiplication instructions \(integer\)](#): multiply-accumulate instructions for integer matrix tiles
- [Matrix-multiplication instructions \(floating-point\)](#): multiply-accumulate instructions for floating-point matrix tiles
- [Load/store instructions for matrix tiles](#): two-dimensional load/store instructions for moving data between memory and vector registers interpreted as matrix tiles
- [Transposing load/store instructions for matrix tiles](#): two-dimensional transposing load/store instructions for moving data between memory and vector registers interpreted as matrix tiles

36.2.1. Matrix tile multiplication geometry

The geometry of the multiplier and the tiles is defined by the new parameter **lambda** (λ) which is encoded in 3 bits in the **vtype** CSR, and vector operation parameters like the widening **W** of the multiplication encoded in the instruction, **LMUL**, **SEW** and **VLEN**.

Matrix tiles are represented using the existing RISC-V V register file and its configuration state. The three matrices in the multiply-accumulate operation $C \leftarrow A \times B^T + C$ are stored as follows:

- The *accumulator* C is stored in a vector register group with element width SEW. Its register group multiplier **MUL_C** is determined by the tile geometry: $\text{MUL_C} = (\text{VLEN} / \text{SEW}) / \lambda^2$, where λ is the tile-layout parameter decoded from the **lambda[2:0]** field in **vtype**. The C register group may start at any vector register index that is **MUL_C**-aligned. $\text{MUL_C} \in \{1, 2, 4, 8, 16\}$. If **MUL_C** = 16, the only allowed vector register indices are 0 and 16.
- The *input matrices* A and B are stored in vector register groups with element width determined by the

instruction: equal to SEW for non-packing variants, SEW/2 for double-packing, SEW/4 for quad-packing, and SEW/8 for octo-packing variants. The effective K dimension of the multiply (K_{eff}) equals $\lambda \times W \times \text{LMUL}$, where W is 1 for non-packing, 2 for double-packing, 4 for quad-packing, and 8 for octo-packing instructions. LMUL scales the A and B register groups along the K dimension only and does not affect C. Only integer values of LMUL are supported by the Zvwm family of Integrated Matrix extensions: $\text{LMUL} \in \{1, 2, 4, 8\}$. Fractional LMUL settings ($\text{LMUL} < 1$) are reserved and shall raise an illegal-instruction exception when used with any IME instruction.

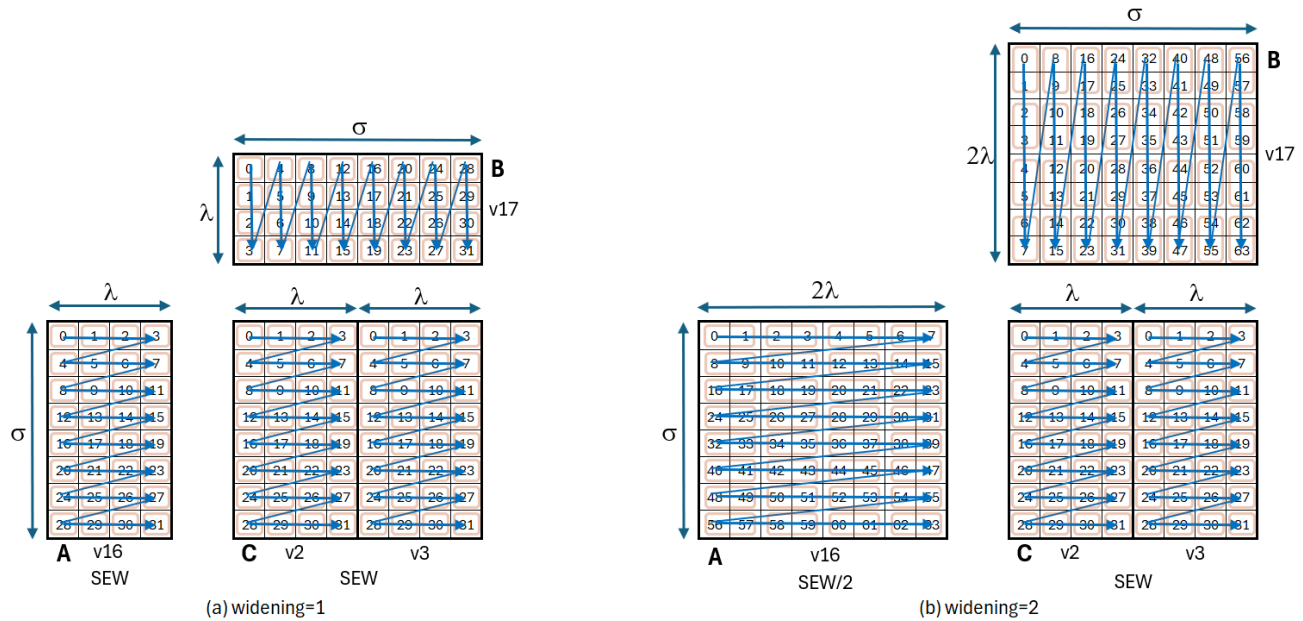


Figure 8. Geometry of matrix tiles and element ordering for 32 element vector registers and $\lambda=4$. VRs are interpreted as 2D tiles. Vector element indices show the tile element order. (a) Non-widening case with A, B, C having the same SEW. (b) Widening case with A and B having half the SEW of C (double-packing).

The K-dimension of the multiplication (shared inner dimension of A and B^T) is determined by λ from **vtype**, scaled by a per-instruction widening factor W and further multiplied by LMUL:

$$K_{\text{eff}} = \lambda \times W \times \text{LMUL}$$

A and B tiles have elements of width $\text{SEW} \div W$; the accumulator C has elements of width SEW (see table below). When operating on integer data types, the signedness of each input is controlled independently by **altfmt_A** and **altfmt_B** (0 = signed, 1 = unsigned; see Section 36.4.3); the accumulator C is always signed. Integer accumulation wraps modulo 2^{SEW} .

The tile dimensions follow from the widening and LMUL settings:

$$\begin{aligned} L_A &= \text{LMUL} \times \text{VLEN} \div (\text{SEW} \div W) && \text{(total A or B tile elements across LMUL registers)} \\ K_{\text{eff}} &= \lambda \times W \times \text{LMUL} \\ M = N &= L_A \div K_{\text{eff}} && \text{(rows = columns of the square accumulator tile)} \end{aligned}$$

The accumulator register group has cardinality $\text{MUL_C} = \text{VLEN} \div (\text{SEW} \times \lambda^2)$, independent of both W and LMUL.

The elements in the vector registers are contiguous in the λ direction, as depicted in Figure 8. Tile A and C elements are sorted in row-major order while tile B^T elements are sorted in column-major order. This choice allows the implementation of the matrix tile multiplication as inner product (eg. a systolic array) or outer product and simplifies the implementation of high-rank updates based outer products.

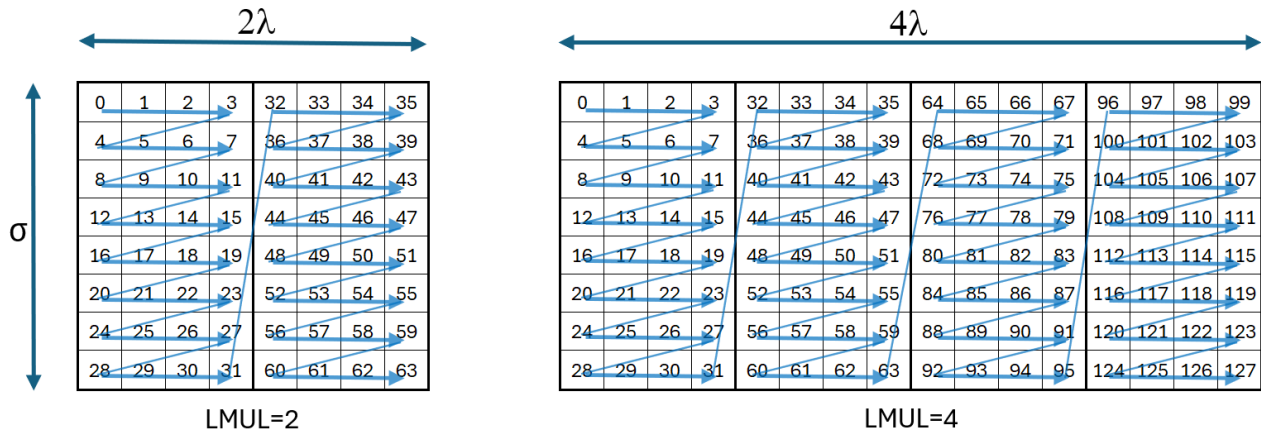


Figure 9. LMUL scaling of matrix tiles for LMUL=2 (left) and LMUL=4 (right).

LMUL scales the tile along the K dimension only, increasing the effective K dimension of the tile by a factor of LMUL. The blue arrows indicate the contiguous elements in the vector register groups that form the tiles in the ordering of element-wise 1D vector operations.

36.3. Subextensions of Zvwm

36.3.1. Naming conventions

The Zvwm family uses two levels of naming.

Family-level extensions group related instructions and are named by function: **Zvwm** (integer multiply-accumulate), **Zvvfmm** (floating-point multiply-accumulate), **Zvwmtps** (tile load/store), and **Zvwmttls** (transposing tile load/store). Implementing a family-level extension requires support for all instructions in that family.

Individual subextensions identify a specific input/output type combination and follow the scheme **Zvv** + *input-type* + *output-type* + **mm**, where the output-type is omitted when it is the same as the input type. The type tokens are:

- Integer: **i4**, **i8**, **i16**, **i32**, **i64**
- Floating-point: **ofp4**, **ofp8**, **fp16**, **bf16**, **fp32**, **fp64**
- Microscaling (E8M0-scaled, block size 32): prefix **x** — e.g. **xofp8**, **xi8**
- Microscaling (E8M0-scaled, block size 16): prefix **xn** — e.g. **xnofp8**, **xni8**

Examples: **Zvvi8i32mm** (Int8 inputs, Int32 accumulator), **Zvvfp16fp32mm** (FP16 inputs, FP32 accumulator), **Zvvxofp8fp16mm** (MXFP8 inputs with BS=32, FP16 accumulator). When the accumulator type matches the input type (e.g. **Zvvi16mm**, **Zvvfp32mm**), the output token is dropped.

36.3.2. Computational subextensions

Table 72 lists the computational subextensions in the Zvwm family which are not related to microscaling. The subextensions specific to microscaling are listed separately in Section 36.12.

Table 72. Computational subextensions in the Zvvm family of Integrated Matrix extensions.

Extension	Dependencies	Multiplicand Types	Accumulator Type
Zvvi4i8mm	Zve32x	[U]Int4, [U]Int4	Int8
Zvvi4i16mm	Zve32x	[U]Int4, [U]Int4	Int16
Zvvi4i32mm	Zve32x	[U]Int4, [U]Int4	Int32
Zvvi8mm	Zve32x	[U]Int8, [U]Int8	Int8
Zvvi8i16mm	Zve32x	[U]Int8, [U]Int8	Int16
Zvvi8i32mm	Zve32x	[U]Int8, [U]Int8	Int32
Zvvi8i64mm	Zve64x	[U]Int8, [U]Int8	Int64
Zvvi16mm	Zve32x	[U]Int16, [U]Int16	Int16
Zvvi16i32mm	Zve32x	[U]Int16, [U]Int16	Int32
Zvvi16i64mm	Zve64x	[U]Int16, [U]Int16	Int64
Zvvi32mm	Zve32x	[U]Int32, [U]Int32	Int32
Zvvi32i64mm	Zve64x	[U]Int32, [U]Int32	Int64
Zvvi64mm	Zve64x	[U]Int64, [U]Int64	Int64
Zvvofp4ofp8mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	OFP8
Zvvofp4fp16mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	IEEE binary16
Zvvofp4bf16mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	BFloat16
Zvvofp4fp32mm	Zve32f	OFP4 (E2M1), OFP4 (E2M1)	IEEE binary32
Zvvofp8mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	OFP8
Zvvofp8fp16mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	IEEE binary16
Zvvofp8bf16mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	BFloat16
Zvvofp8fp32mm	Zve32f	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	IEEE binary32
Zvvofp8fp64mm	Zve64d	OFP8 (E4M3 or E5M2), OFP8 (E4M3 or E5M2)	IEEE binary64
Zvvfp16mm	Zve32f	IEEE binary16, IEEE binary16	IEEE binary16
Zvvfp16fp32mm	Zve32f	IEEE binary16, IEEE binary16	IEEE binary32
Zvvfp16fp64mm	Zve64d	IEEE binary16, IEEE binary16	IEEE binary64
Zvvbf16mm	Zve32f	BFloat16, BFloat16	BFloat16
Zvvbf16fp32mm	Zve32f	BFloat16, BFloat16	IEEE binary32
Zvvbf16fp64mm	Zve64d	BFloat16, BFloat16	IEEE binary64

Extension	Dependencies	Multiplicand Types	Accumulator Type
Zvvp32mm	Zve32f	IEEE binary32, IEEE binary32	IEEE binary32
Zvvp32fp64mm	Zve64d	IEEE binary32, IEEE binary32	IEEE binary64
Zvvp64mm	Zve64d	IEEE binary64, IEEE binary64	IEEE binary64

36.4. New fields in the Vector Type (**vtype**) Register

The Zvmm family of Integrated Matrix extensions defines the following additional fields in the **vtype** CSR:

- **lambda[2:0]** at **vtype[XLEN-2:XLEN-4]** is the Selected Lambda
- **altfmt_A** at **vtype[XLEN-5]** is the Alternate Format for input A
- **altfmt_B** at **vtype[XLEN-6]** is the Alternate Format for input B
- **bs** at **vtype[XLEN-7]** is the Block Size selector for microscaling

36.4.1. Selected lambda (**lambda[2:0]**) field

The **lambda[2:0]** field is a 3-bit WARL read/write field in the **vtype** CSR that holds the dynamic *selected lambda* — the tile-layout parameter that determines the geometry of the accumulator tile C and, together with W and LMUL, the effective K dimension $K_{\text{eff}} = \lambda \times W \times \text{LMUL}$. It governs both multiply-accumulate and 2D load/store instructions.

lambda[2:0] is a WARL field, so it can only be programmed to values supported by the implementation. One discovery mechanism is to attempt to program different values and read back which are retained.

Table 73. *lambda[2:0]* (selected lambda) encoding

lambda[2:0]			K
0	0	0	invalid/dynamic
0	0	1	1
0	1	0	2
0	1	1	4
1	0	0	8
1	0	1	16
1	1	0	32
1	1	1	64

36.4.2. Alternate floating-point format for the output (**altfmt**)

The Zvmm Integrated Matrix floating-point extensions use the *altfmt* field (as defined by Zvfbfa) to select the floating-point format of the elements in the output accumulator matrix C, in conjunction with **vsew**.

Table 74. Output data type encoding (*altfmt* × *vsew*[2:0])

altfmt	vsew[2:0]			FP format
0	0	0	0	E4M3 (OFP8)
1	0	0	0	ESM2 (OFP8)

altfmt	vsew[2:0]			FP format
0	0	0	1	IEEE binary16
1	0	0	1	BFloat16
0	0	1	0	IEEE binary32
1	0	1	0	<i>reserved</i>
0	0	1	1	IEEE binary64
1	0	1	1	<i>reserved</i>
X	1	X	X	<i>reserved</i>

36.4.3. Alternate formats for inputs (**altfmt_A**, **altfmt_B**)

The **altfmt_A** and **altfmt_B** fields are 1-bit read/write fields added to the **vtype** CSR by the Zvwm family of Integrated Matrix extensions. They select the interpretation of elements in input matrices A and B, respectively. Their meaning depends on whether the instruction is a floating-point or integer multiply-accumulate.

The **altfmt_A** and **altfmt_B** bits are located at **vtype**[XLEN-5] and **vtype**[XLEN-6], immediately below the **lambda**[2:0] field. These positions are outside the **vsetvli** immediate field and must be configured via **vsetvli** (with the full **vtype** value in a register) or **vsetivli**.

36.4.3.1. Floating-point instructions

For floating-point multiply-accumulate instructions, **altfmt_A** and **altfmt_B** select the floating-point format of the corresponding input operand. These fields are only significant for sub-word floating-point input operands (OFP8, 16-bit FP); for IEEE binary32 and IEEE binary64 inputs they are ignored.

Table 75. FP input data type encoding (*altfmt_A* or *altfmt_B* × input element width)

altfmt_A / altfmt_B	Input element width	FP format
0	4-bit	E2M1 (OFP4)
1	4-bit	<i>reserved</i>
0	8-bit	E4M3 (OFP8)
1	8-bit	E5M2 (OFP8)
0	16-bit	IEEE binary16
1	16-bit	BFloat16

36.4.3.2. Integer instructions

For integer multiply-accumulate instructions, **altfmt_A** and **altfmt_B** select whether the corresponding input operand is interpreted as signed or unsigned.

Table 76. Integer input signedness encoding (*altfmt_A* or *altfmt_B*)

altfmt_A / altfmt_B	Interpretation
0	Signed
1	Unsigned

36.4.4. Microscaling block size (bs)

The **bs** field is a 1-bit read/write field in the **vtype** CSR that selects the block size used for microscaling operations ($vm=0$). When $vm=1$, the **bs** field is ignored.

Table 77. Block size encoding

bs	Block size
0	32 elements
1	16 elements

The **bs** bit is located at **vtype**[XLEN-7], immediately below **altfmt_B**. Like **altfmt_A** and **altfmt_B**, this position is outside the **vsetvli** immediate field.

36.5. Storage formats

36.5.1. Element packing in input tiles

For non-widening multiply-accumulate instructions ($W=1$), the input elements A and B have the same width as the accumulator (SEW) and no packing occurs.

For widening instructions ($W=2$ or $W=4$), each input register group holds elements at the effective input element width $EEW = SEW \div W$. Because tile load instructions always transfer data at SEW granularity, every loaded SEW-bit position contains W contiguous narrow elements that the multiply-accumulate instruction consumes as a sub-dot-product.

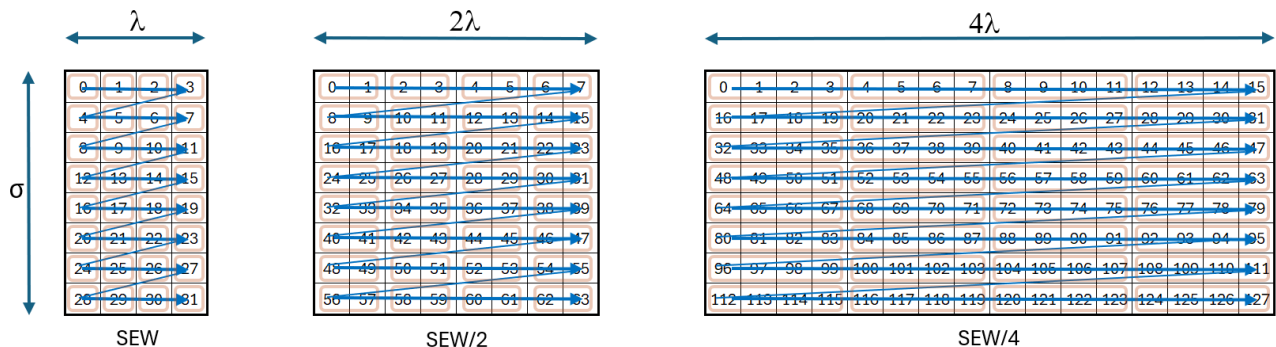


Figure 10. Element distribution and tile geometry example for $L=32$, SEW wide elements (left), two SEW/2 wide elements packed per SEW (middle), and four SEW/4 wide elements per SEW (right). Packing/widening by W increases the effective K dimension of the tile by a factor of W .

36.5.1.1. Byte-sized and wider elements ($EEW \geq 8$)

When the input element width is 8 bits or wider ($EEW \in \{8, 16, 32, 64\}$), the elements within the input register group follow standard RISC-V V element ordering: element k at width EEW occupies bits $[k \times EEW + EEW - 1 : k \times EEW]$ of the register group. This is identical to the layout produced by a vector load at the same element width (e.g., **vle8.v** for $EEW=8$) and requires no special data preparation beyond the tile load itself.

36.5.1.2. Sub-byte elements ($EEW = 4$)

Four-bit input elements (OFP4 (E2M1), Int4) are packed two per byte in little-endian nibble order:

- The *even-indexed* element (element $2n$) occupies the **lower** nibble [3:0] of byte n .
- The *odd-indexed* element (element $2n + 1$) occupies the **upper** nibble [7:4] of byte n .

This extends the standard RISC-V V element-indexing convention to sub-byte widths: element k at width 4 occupies bits $[4k + 3 : 4k]$ of the register group, consistent with the formula for all element widths.

Because tile loads operate at SEW (≥ 8 bits), each loaded byte already contains a naturally ordered pair of 4-bit elements. Software preparing input data in memory must pack adjacent elements within each byte accordingly.

36.6. Zvmm: Extension for matrix multiplication on vector registers interpreted as 2D integer matrix tiles

The **Zvmm** family of extensions provides instructions that perform matrix multiply-accumulate on integer data, computing $C \leftarrow C + A \times B^T$, where A, B, and C are matrix tiles held in the vector register file.

The following table lists the integer matrix tile multiplication instructions. The arguments are:

- **vd**: Destination vector register group containing the C matrix tile.
- **vs1**: Source vector register (group) containing the A matrix tile.
- **vs2**: Source vector register (group) containing the B^T matrix tile.

Mnemonic	W	A/B element width	C element width
vmacc.vv vd, vs1, vs2	1	SEW	SEW
vwmacc.vv vd, vs1, vs2	2	SEW/2	SEW
vqwmacc.vv vd, vs1, vs2	4	SEW/4	SEW
v8wmacc.vv vd, vs1, vs2	8	SEW/8	SEW

Vector masking is not supported on matrix multiply-accumulate instructions: the **vm** bit in the encoding must be 1. For **vmacc.vv**, **vm=0** is *reserved*. For **vwmacc.vv**, **vqwmacc.vv**, and **v8wmacc.vv**, **vm=0** encodes **vfwmacc.vv**, **vfqwmacc.vv**, and **vf8wmacc.vv** respectively — integer-input, floating-point-accumulate, E8M0-microscaled instructions (see [Section 36.7.3](#) and [Section 36.13.3](#)). For floating-point multiply-accumulate instructions, **vm=0** enables microscaling (see [Section 36.7.3](#)).

36.6.1. Arithmetic Considerations

36.6.1.1. Integer accumulation

For integer multiply-accumulate instructions, all intermediate results are reduced modulo 2^{SEW} . Because modular addition is both associative and commutative, the final result is uniquely defined regardless of the accumulation order or grouping factor.

36.7. Zvffmm: Extension for matrix multiplication on vector registers interpreted as 2D floating-point matrix tiles

The **Zvffmm** family of extensions provides floating-point matrix multiply-accumulate instructions, computing $C \leftarrow C + A \times B^T$. A, B, and C are matrix tiles held in the vector register file.

The following table lists the integer matrix tile multiplication instructions. The arguments are:

- **vd**: Destination vector register group containing the C matrix tile.
- **vs1**: Source vector register (group) containing the A matrix tile.
- **vs2**: Source vector register (group) containing the B^T matrix tile.
- **v0.scale**: Block scales for A and B^T operands in MX-scaling mode.

Mnemonic	W	A/B element width	C element width	Remark
vfmmacc.vv vd, vs1, vs2[, v0.scale]	1	SEW	SEW	floating point and MX scaled A/B tiles
vfwmacc.vv vd, vs1, vs2[, v0.scale]	2	SEW/2	SEW	floating point and MX scaled A/B tiles
vfqwmacc.vv vd, vs1, vs2[, v0.scale]	4	SEW/4	SEW	floating point and MX scaled A/B tiles
vf8wmacc.vv vd, vs1, vs2[, v0.scale]	8	SEW/8	SEW	floating point and MX scaled A/B tiles
vfwmimacc.vv vd, vs1, vs2, v0.scale	2	SEW/2	SEW	only MX-scaled integer A/B tiles
vfqwmimacc.vv vd, vs1, vs2, v0.scale	4	SEW/4	SEW	only MX-scaled integer A/B tiles
vf8wmimacc.vv vd, vs1, vs2, v0.scale	8	SEW/8	SEW	only MX-scaled integer A/B tiles

Vector masking is not supported on matrix multiply-accumulate instructions: the **vm** bit in the encoding selects between normal, unscaled arguments (**vm=1**) and microscaled MX^* matrix tile arguments. For details see [Section 36.7.3](#).

The floating-point format of each operand tile is controlled by fields in **vtype**:

- **altfmt_A** and **altfmt_B** select the FP format for A and B input elements, interpreted in conjunction with the input element width $SEW \div W$ (see [Section 36.4.3](#)). The A and B input tiles may use different formats (**altfmt_A** $\neq$ **altfmt_B**); see [Section 36.7.1](#) for subextension requirements.
- **altfmt** (as defined by **Zvfbfa**) selects the FP format for the C accumulator elements, interpreted in conjunction with SEW (see [Section 36.4.2](#)).

36.7.1. Mixed-format inputs (**altfmt_A** $\neq$ **altfmt_B**)

Mixed-format inputs (**altfmt_A** $\neq$ **altfmt_B**) are permitted for all floating-point multiply-accumulate instructions. Each $A \times B$ product is computed in sufficient internal precision and rounded to the C accumulator format; the two input formats need not match. The valid mixed-format combinations are listed in [Table 84](#).

36.7.1.1. Mixed-format multiplication semantics

Each $A \times B$ element product is computed per IEEE 754: the exact (infinite-precision) mathematical product is formed, then rounded once to the C accumulator format using the dynamic rounding mode from **frm**. This definition applies uniformly regardless of whether **fmt_A** equals **fmt_B**.

The exact product of two floating-point values with p_A and p_B significand bits has at most $p_A + p_B$ significand bits. An implementation may compute the product in any internal format with at least that many significand bits and then round once to **fmt_C**.

Table 78. Significand widths (p) of supported formats

Format	p (significand bits)
E2M1 (OFP4)	2
E4M3 (OFP8)	4
E5M2 (OFP8)	3
IEEE binary16	11
BFloat16	8
IEEE binary32	24
IEEE binary64	53

When $p_A + p_B \leq p_C$, the product is exact in **fmt_C** and no rounding is needed. This holds for all mixed-format widening combinations — for example, $E4M3 \times E5M2 \rightarrow FP16$ ($4 + 3 = 7 \leq 11$) and $FP16 \times BF16 \rightarrow FP32$ ($11 + 8 = 19 \leq 24$).

An implementation need not use a standard IEEE 754 type for the internal product; any representation that captures the exact product (e.g., an integer significand–exponent pair) suffices, provided the final result equals the correctly-rounded value in **fmt_C**.

The subextension requirements for mixed-format operation depend on the input element width:

36.7.1.2. Sub-word inputs (OFP4, OFP8)

For 4-bit inputs, only OFP4 (E2M1) is supported (**altfmt_A** = **altfmt_B** = 0). The encoding **altfmt_A** = 1 or **altfmt_B** = 1 is reserved for 4-bit inputs.

Likewise, for 8-bit inputs, **altfmt_A** and **altfmt_B** independently select E4M3 or E5M2. All four combinations are covered by the same OFP8 subextension for the given output format.

Mixed OFP8 inputs ($E4M3 \times E5M2$) are only permitted with widening instructions (**vfwmacc.vv**, **vfqwmacc.vv**), not with **vfmacc.vv** (OFP8 $\rightarrow$ OFP8), because the exact product (up to 7 significand bits) exceeds the OFP8 output precision ($p \leq 4$).

36.7.1.3. 16-bit inputs (IEEE binary16, BFloat16)

For 16-bit inputs, **altfmt_A** and **altfmt_B** independently select IEEE binary16 or BFloat16. When both are the same, a single subextension suffices (Zvvp16mm* for IEEE binary16, Zvvpbf16mm* for BFloat16), and all three instructions (**vfmacc.vv**, **vfwmacc.vv**, **vfqwmacc.vv**) are available.

When **altfmt_A** $\neq$ **altfmt_B** (one input is IEEE binary16 and the other is BFloat16), only widening instructions are permitted — **vfmacc.vv** with mixed FP16/BF16 inputs raises an illegal-instruction exception because the exact product (up to 19 significand bits) exceeds the precision of any 16-bit output format.

For the permitted widening cases, both the IEEE binary16 and BFloat16 subextensions for the same instruction and output format are required. The instruction is legal only when both subextensions are present; if either is missing, the instruction raises an illegal-instruction exception.

Table 79. Subextension requirements for mixed FP16/BF16 inputs

Instruction	Output format	Required subextensions (both must be present)
vfwmacc.vv	FP32 (SEW=32)	Zvvp16fp32mm AND Zvvpbf16fp32mm
vfqwmacc.vv	FP64 (SEW=64)	Zvvp16fp64mm AND Zvvpbf16fp64mm

If the required subextensions are not both present, the instruction raises an illegal-instruction exception.

36.7.1.4. 32-bit and wider inputs

For IEEE binary32 and IEEE binary64 inputs, `altfmt_A` and `altfmt_B` are ignored (there is only one format per width), so mixed-format operation does not apply.

All multiply and add operations use the dynamic rounding mode from `frm`; floating-point exception flags are accumulated into `fflags`.

The K-dimension, tile-dimension formulae, `MUL_C`, and instruction-to-widening-factor mapping are the same as for the integer family (see [Matrix-multiplication instructions \(integer\)](#)); the floating-point mnemonics are `vfwmacc.vv` (W=1), `vfwmacc.vv` (W=2), and `vfqwmacc.vv` (W=4).

As with the integer multiply-accumulate instructions, vector masking is not supported. When `vm=0`, the instruction does not apply a vector mask; instead, `v0` supplies paired E8MO block-scale factors for microscaling operation (see [Section 36.7.3](#)).

36.7.1.5. Mixed-format inputs arithmetic considerations

Whenever mixed formats are used in the computational instructions, one must follow the guidelines in [Section 36.7.2](#).

36.7.2. Arithmetic considerations

Each multiply-accumulate instruction computes, for every output element $C_{m,n}$:

$$C_{m,n} \leftarrow C_{m,n} + \sum_{k=0}^{K_{\text{eff}}-1} A_{m,k} \times B_{k,n}$$

where $K_{\text{eff}} = \lambda \times W \times \text{LMUL}$. This section specifies how the K_{eff} product terms may be grouped and when intermediate rounding is permitted.

36.7.2.1. Sub-dot-products

For widening instructions ($W = 2$ or $W = 4$), the W narrow input-element pairs packed within one accumulator-width (SEW-bit) position form a natural computational unit called a *sub-dot-product*. The W multiplications of (SEW÷ W)-bit elements are performed and their products summed.

Each product of two (SEW÷ W)-bit values is *exact* at SEW precision: the significand has at most $2 \times p_{\text{input}} - 1$ bits, which fits in the wider SEW format. The sub-dot-product can therefore be computed with very little loss at SEW or wider precision.

There are $K_{\text{eff}} \div W = \lambda \times \text{LMUL}$ sub-dot-products per output element.

For non-widening instructions ($W = 1$), each product of two SEW-bit values is exact at $2 \times \text{SEW}$ bits; a sub-dot-product consists of a single product term.

36.7.2.2. Accumulation and rounding model (floating-point)

The resulting value of element $C_{m, n}$ for any given LMUL must match the value computed by applying a series of multiply-accumulate instructions with LMUL=1, in increasing order of vector register indices. Therefore, it is enough to specify the result of the computations

$$C_{m, n} \leftarrow C_{m, n} + \sum_{k=0}^{\lambda-1} A_{m, k} \times B_{k, n}$$

An implementation partitions the λ sub-dot-products for each output element into consecutive groups of G sub-dot-products.

- Within each group, the G partial results are accumulated into a partial sum S using internal precision that requires no rounding to SEW precision inside a group.
- After each group, the accumulated partial sum S is *added to the running value of $C_{m, n}$* by computing

$$C_{m, n} \leftarrow \text{round}_{frm}(C_{m, n} + (\text{round}_{rto}(S))),$$

where the accumulation rounding is performed with the rounding mode from **frm**. The rounding of partial sum S *before* it is accumulated to the running value of $C_{m, n}$ is optional and, if performed, must use round-to-odd (rto) mode.

The value of G is *implementation-defined* and may depend on SEW, W , λ , LMUL, and the microarchitecture. It must satisfy:

- G is a power of two;
- $1 \leq G \leq \lambda$.

The resulting number of rounding operations per output element of C is $[2](\lambda \times \text{LMUL}) \div G$, where the factor of 2 is present if the accumulated partial sum is rounded before being added to the running value.



*In a **systolic-array** datapath, G is typically 1: each sub-dot-product is rounded and added to C immediately, yielding $\lambda \times \text{LMUL}$ rounding additions per output element.*

*In an **outer-product** datapath, G is typically on the order of λ (e.g. λ , $\lambda \div 2$, $\lambda \div 4$, or $\lambda \div 8$): multiple sub-dot-products are accumulated at extended internal precision before a single rounding and C addition. This significantly reduces the number of expensive full-precision additions.*

Software must not depend on a particular value of G .

It is expected an implementation will have to disclose its chosen value of G in order to be certified as compliant.

The rounding of partial sum S is made optional in order to match established practices in industry and other RISC-V matrix extensions.

Because G is implementation-defined, two conforming implementations may produce floating-point results that differ in the least-significant bits for identical inputs. Bit-exact reproducibility of floating-point matrix multiply-accumulate results across different implementations is therefore *not* guaranteed.

Floating-point exception flags (inexact, overflow, underflow, invalid, etc.) are accumulated into **fflags**; the order in which individual exceptions are raised within a single instruction execution is implementation-defined.



Editorial Note

We could ammend the requirement that "... the G partial results are accumulated into a partial sum S using internal precision that requires no rounding to SEW precision inside a group" with the license to use the Bulk Normalization of Dot-Products (BNDP) procedure. If we do that, we must require the implementor to disclose the choice of BNDP parameters. Adding the license would result in more compatibility with other RISC-V matrix extensions.

36.7.3. Microscaling support (`v0.scale`)

Microscaling floating-point formats (as defined by the *OCF Microscaling Formats (MX) Specification* ^[1]) represent tensors using narrow per-element values (e.g., OFP4 or OFP8) combined with a coarser shared scale factor applied to a *block* of consecutive elements along the K dimension. The standard OCF MX block size is 32 elements; however, recent research (Lee et al., "MX+: A Robust Framework for Finer-Grained MX Quantization", MICRO '25) demonstrates that a block size of 16 significantly improves accuracy for activation tensors with outliers.

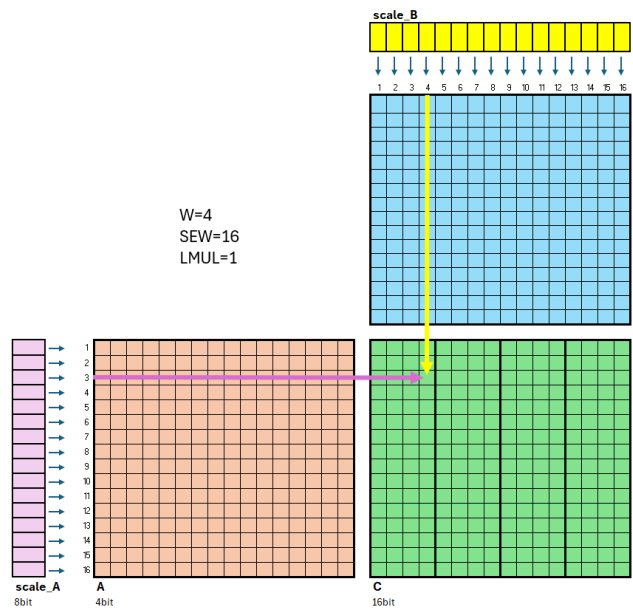


Figure 11. Block scaling multiplication example with $SEW=16$, $\lambda=4$, $W=4$, $LMUL=1$. Block scales are applied along A rows and B columns.

The Integrated Matrix extensions support microscaling on all floating-point multiply-accumulate instructions by encoding `vm=0` in the instruction and supplying paired E8MO block-scale factors through `v0`. The `bs` field in `vtype` selects the block size (32 or 16 elements).

The assembly syntax for a microscaled operation appends `v0.scale` as the final operand, analogous to `v0.t` for masked operations in the base V extension:

```
vfmacc.vv    vd, vs1, vs2, v0.scale
vfwmacc.vv   vd, vs1, vs2, v0.scale
vfqwmacc.vv  vd, vs1, vs2, v0.scale
```

When `vm=0` is not specified (i.e., `vm=1`, the default), no scaling is applied, `v0` is not read, and the `bs` field is ignored.

36.7.3.1. Semantics

When microscaling is active (`vm=0`), each output element is computed as:

$$C_{m,n} \leftarrow C_{m,n} + \sum_{s=0}^{S-1} (\text{scale_A}_{m,s} \times \text{scale_B}_{n,s} \times (\sum_{k \in \text{blocks}} A_{m,k} \times B_{k,n}))$$

where $S = \lceil K_{\text{eff}} / \text{block_size} \rceil$ is the number of scale blocks per row/column, and block s covers elements $[s \times \text{block_size}, \min((s+1) \times \text{block_size}, K_{\text{eff}}) - 1]$.

$\text{scale_A}_{m,s}$ and $\text{scale_B}_{n,s}$ are power-of-two values decoded from 8-bit E8MO fields in **v0**. The E8MO format uses an 8-bit biased exponent (bias 127) representing values from 2^{-127} to 2^{127} ; the bit pattern 0xFF encodes NaN.

The per-block scales for A and B are applied as exact power-of-two multiplications (implemented as exponent additions) and therefore introduce no rounding error. If any $\text{scale_A}_{m,s}$ or $\text{scale_B}_{n,s}$ is NaN (0xFF), the corresponding output element $C_{m,n}$ is set to the default NaN regardless of the input element values.

When a non-NaN E8MO value represents a power-of-two that overflows the accumulator FP format (e.g. 2^{128} cannot be represented in FP16 which has a maximum finite exponent of 2^{15}), the converted scale value is $+\infty$. The subsequent block-scale multiply follows IEEE 754 rules: $+\infty$ multiplied by a non-zero finite value yields $\pm\infty$ (propagating into the accumulator via **fp_add**); $+\infty$ multiplied by zero yields the default NaN, which sets the output element to NaN via the NaN propagation rule above. Implementations shall handle this case as specified — not as an implementation-defined result.

36.7.3.2. Scale layout in v0

The register **v0** holds paired E8MO scale values at an effective element width of 16 bits. Each 16-bit element contains one (scale_A, scale_B) pair:

- Lower byte [7:0]: **scale_A** — the E8MO scale for a row of A
- Upper byte [15:8]: **scale_B** — the E8MO scale for a column of B

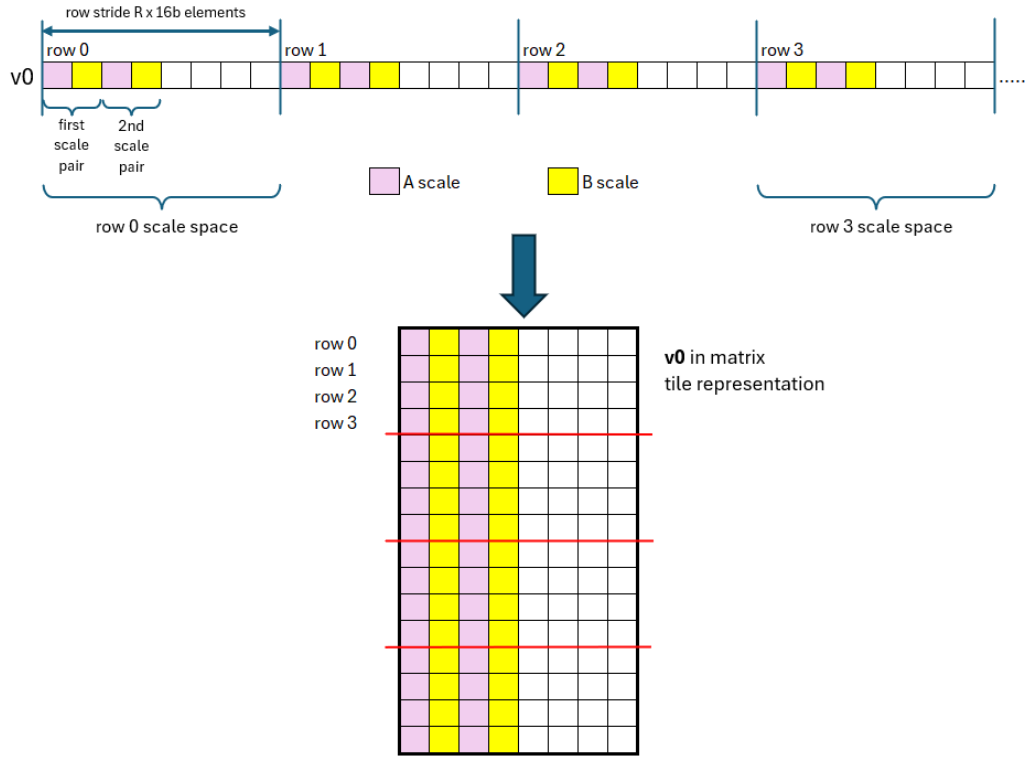


Figure 12. Block scales are folded into **v0** in pairs, separated by the row stride $R = \lambda \times \text{SEW} / 16$. Each block-scale pair is used for **bs** consecutive tile elements in the corresponding *A* row / B^T column. In matrix-tile representation **v0** has the paired scales arranged in the correct tile rows. In a multi-lane setup the scales are distributed across the lanes appropriately.

The row stride in **v0** is $R = \lambda \times \text{SEW} \div 16$ in units of 16-bit elements (see Figure 12). Within each row, $S = \lceil K_{\text{eff}} / \text{block_size} \rceil$ elements are active. Element index $p = m \times R + s$ addresses the scale pair for row (or column) m at block s . Specifically:

- **scale_A_{m, s}** is the lower byte of the 16-bit element at index $(m \times R + s)$
- **scale_B_{n, s}** is the upper byte of the 16-bit element at index $(n \times R + s)$

The total number of 16-bit elements spanned per tile dimension is $M \times R$, where $M = N = \text{VLEN} \div (\text{SEW} \times \lambda)$. Because $M \times R = (\text{VLEN} \div (\text{SEW} \times \lambda)) \times (\lambda \times \text{SEW} \div 16) = \text{VLEN} \div 16$, the scale data fills exactly one register regardless of configuration.

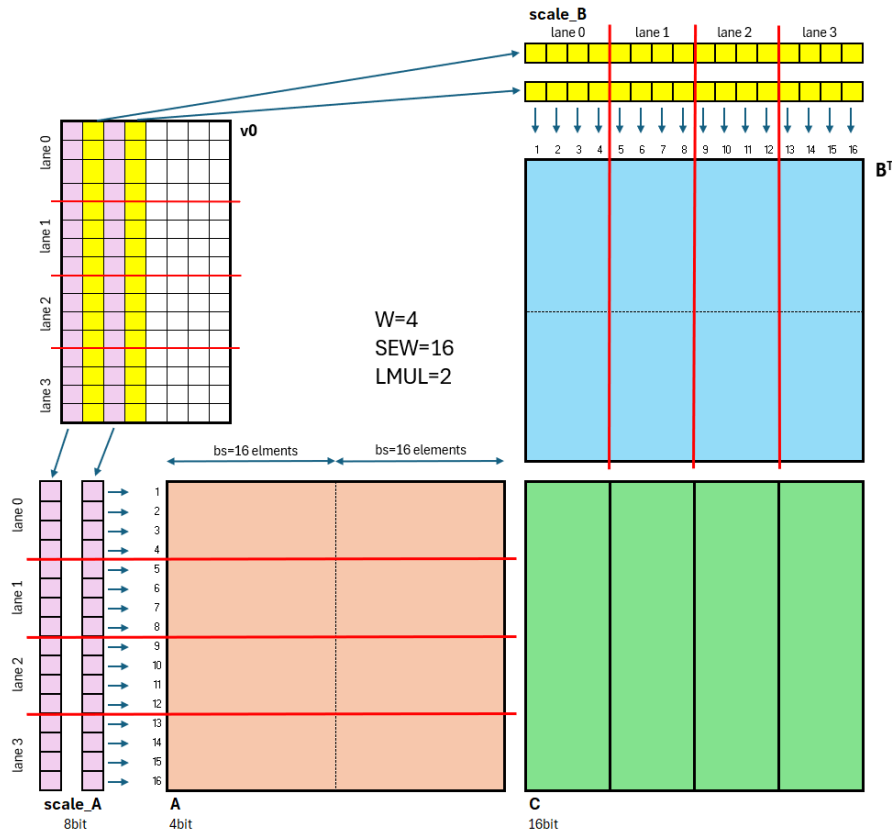


Figure 13. Example for the use of microscales in a 4-lane setup for two blocks per row / column. The red lines symbolize the split of the VRs across the lanes. The R-strided scales distribution in **v0** leads to the correct lane-wise block scales layout.

Capacity proof:

$$M \times R = (VLEN \div (SEW \times \lambda)) \times (\lambda \times SEW \div 16) = VLEN \div 16.$$

Each of the $VLEN \div 16$ positions is a 16-bit element, so the total is exactly VLEN bits — the scale data fills one register in every legal configuration.

R ≥ S proof (tile stride is never smaller than the active count):

- **block_size = 32:** The encoding tables (Table 84, Section 36.13.3) mark all combinations with $W \times LMUL > 2 \times SEW$ as *reserved*, so every legal encoding satisfies $W \times LMUL \leq 2 \times SEW$. Therefore $K_{eff} = \lambda \times W \times LMUL \leq 2 \times \lambda \times SEW$, giving $S = \lceil K_{eff} / 32 \rceil \leq \lceil 2 \times \lambda \times SEW / 32 \rceil \leq \lambda \times SEW / 16 = R$.
- **block_size = 16:** The BS=16 legality check ($W \times LMUL \leq SEW$, stated in each instruction's Exceptions section) ensures $K_{eff} = \lambda \times W \times LMUL \leq \lambda \times SEW$, giving $S = \lceil K_{eff} / 16 \rceil \leq \lceil \lambda \times SEW / 16 \rceil \leq \lambda \times SEW / 16 = R$.

When $S < R$, positions $s \in [S, R)$ within each row are padding and their contents are ignored by the instruction.

Table 80. Scale layout parameters (block_size = 32, VLEN = 256)

SEW	λ	W	LMUL	M	S	R	16-bit extent (M×R)
8	2	1	1	16	1	1	16
8	2	1	8	16	1	1	16
8	2	2	1	16	1	1	16

SEW	λ	W	LMUL	M	S	R	16-bit extent (M×R)
8	2	2	8	16	1	1	16
16	1	1	1	16	1	1	16
16	1	1	2	16	1	1	16
16	1	2	1	16	1	1	16
16	1	4	1	16	1	1	16
16	1	4	8	16	1	1	16 [^] [2]
16	2	4	1	8	1	2	16
16	2	4	8	8	2	2	16 [^] [2]
32	1	1	1	8	1	2	16
32	1	2	1	8	1	2	16
32	2	1	1	4	1	4	16
64	1	1	1	4	1	4	16

Within each row stride, elements at positions $s \in [S, R)$ are padding and need not be initialised. When $S = R$, the layout is fully packed with no padding.

36.7.3.3. Block size selection

The **bs** field in **vtype** (see [Section 36.4.4](#)) selects the microscaling block size. When **bs=0**, the block size is 32 elements (standard OCP MX); when **bs=1**, the block size is 16 elements.

Block size 16 doubles the number of scales per row/column (S is doubled), which doubles the $M \times S$ capacity requirement. To guarantee that the paired scales fit within **v0**, the following constraint is imposed when **bs=1** and **vm=0**:

$$W \times LMUL \leq SEW$$

If this constraint is violated, the instruction raises an illegal-instruction exception.

Table 81. Maximum LMUL for block_size = 16

Input format	Instruction	W	SEW	Max LMUL
FP4 → FP16/BF16	vfqwmacc	4	16	4
FP4 → OFP8	vfwmacc	2	8	4
Int4 → FP16/BF16	vfqwimacc	4	16	4
All others	—	—	—	8 (no restriction)

36.7.3.4. Applicability

Microscaling via **v0.scale** is supported on the widening floating-point multiply-accumulate instructions (**vfwmacc.vv**, **vfqwmacc.vv**, **vf8wmacc.vv**) and on the integer-input FP-accumulate instructions **vfwmacc.vv** (MXINT8→FP16/BF16), **vfqwimacc.vv** (MXINT4→FP16/BF16, MXINT8→FP32), and **vf8wimacc.vv** (MXINT4→FP32, MXINT8→FP64). For **vmacc.vv**, **vm=0** remains *reserved* and raises an illegal-instruction exception.

MXINT8 (E8M0-scaled signed 8-bit integer) is defined by the *OCP Microscaling Formats (MX) v1.0 Specification*. MXINT4 (E8M0-scaled signed 4-bit integer) is defined analogously by this specification: each block of 4-bit signed two's-complement elements shares a single E8M0 exponent-only scale factor, following the same block-scaling semantics as MXINT8 but with 4-bit elements.

Microscaling requires $SEW \times \lambda \geq 16$ to ensure that the paired 16-bit scale elements fit within **v0**. Configurations where $SEW \times \lambda < 16$ with **vm=0** raise an illegal-instruction exception.

Microscaling is particularly useful with narrow input formats (OFP4, OFP8) where the limited dynamic range of the per-element format is compensated by the block-level E8M0 exponent, enabling high-throughput inference on microscaling-quantised models.

36.7.3.5. Loading or constructing paired scales

This section is informative (non-normative).

*The tile-strided scale layout in **v0** uses $M \times R$ 16-bit elements (always $VLEN \div 16$ elements, filling exactly one register). Within each of the M row strides of length R , the first S positions are active and positions $[S, R)$ are padding. The following options are listed from simplest to most complex.*

Option 1 — Pre-paired tile-strided load via **vmtl.v** (single instruction):

*When the application stores pre-paired (scale_A, scale_B) 16-bit elements in a 2D scale matrix (M rows, stride $LD \geq R$ between rows), a single **vmtl.v** loads the current tile's scales directly into **v0**:*

```
# a0 = base of pre-paired scale sub-array for current tile
# a2 =  $M \times R$  (=  $VLEN / 16$ )
# a3 =  $LD$  (leading dimension in 16-bit elements,  $LD \geq R$ )
vsetvli t0, a2, e16, m1, ta, ma
vmtl.v  v0, (a0), a3          # tile-load pre-paired scales
```



*With $EEW = 16$ and $LMUL = 1$, linesize = $\lambda = R$ (when $SEW = 16$). Each **vmtl.v** line corresponds to one scale row, and LD allows rows to be non-contiguous in the outer scale matrix. When $LD = R$, this reduces to a unit-stride load.*



*This form requires $SEW = 16$ (i.e., $R = \lambda$). For $SEW > 16$, $R > \lambda$ and a single **vmtl.v** cannot span a full scale row; use Option 2 instead.*

Option 2 — Pre-paired flat load (**vle16.v**):

If the pre-paired tile-strided data is already contiguous in memory ($M \times R$ 16-bit elements), a unit-stride load suffices:

```
# a0 = pointer to contiguous pre-paired tile-strided scale
array
# a2 =  $M \times R$  (=  $VLEN / 16$ )
vsetvli t0, a2, e16, m1, ta, ma
vle16.v v0, (a0)
```

This works for all SEW values since $M \times R = \text{VLEN} \div 16$ always.

Options 3–5 — Constructing from separate **scale_A** / **scale_B** arrays:

*The following sequences construct the paired layout from separate **scale_A** and **scale_B** byte arrays, producing $M \times S$ contiguous pairs. This produces the correct tile-strided layout when $S = R$ (fully packed). When $S < R$, the output must be placed at stride R , e.g., by storing the $M \times S$ pairs into a pre-zeroed $M \times R$ buffer before loading with Option 2.*

Option 3 — Base V (widen + shift + OR):

```
# a0 = &scale_A, a1 = &scale_B (M × S bytes each), a2 = M × S
vsetvli t0, a2, e8, mf2, ta, ma
vle8.v  v1, (a0)           # load scale_A
vle8.v  v2, (a1)           # load scale_B
vsetvli t0, a2, e16, m1, ta, ma
vzext.vf2 v0, v1           # zero-extend scale_A to 16-bit
vzext.vf2 v3, v2           # zero-extend scale_B to 16-bit
vsll.vi v3, v3, 8          # shift scale_B into upper byte
vor.vv  v0, v0, v3         # combine into paired layout
```

Option 4 — Zvbb (widening shift + OR):

```
# a0 = &scale_A, a1 = &scale_B (M × S bytes each), a2 = M × S
vsetvli t0, a2, e8, mf2, ta, ma
vle8.v  v1, (a0)           # load scale_A
vle8.v  v2, (a1)           # load scale_B
vwsll.vi v0, v2, 8         # widen scale_B to 16-bit,
shift to upper byte
vsetvli t0, a2, e16, m1, ta, ma
vzext.vf2 v3, v1           # zero-extend scale_A to 16-bit
vor.vv  v0, v0, v3         # combine into paired layout
```

Option 5 — Zvzip (interleave):

```
# a0 = &scale_A, a1 = &scale_B (M × S bytes each), a2 = M × S
vsetvli t0, a2, e8, mf2, ta, ma
vle8.v  v1, (a0)           # load scale_A
vle8.v  v2, (a1)           # load scale_B
vzip.vv v0, v1, v2         # interleave bytes, EMUL = 2 ×
mf2 = m1
```

*The **vzip.vv** instruction interleaves at SEW=8: the byte sequence $[A[0], B[0], A[1], B[1], \dots]$ reinterpreted as 16-bit little-endian elements yields $\text{element}[i] = B[i] \ll 8 \mid A[i]$, which is exactly the required paired layout.*

Option 6 — Separate scales via **vmtl.v** + **vzip.vv** (SEW = 16):

When $SEW = 16$ and scales reside in separate 2D arrays, `vmtl.v` can load each scale array with the correct tile stride, then `vzip.vv` pairs them:

```
# a0 = &scale_A, a1 = &scale_B (M rows, LD bytes between
rows)
# a2 = M × R (= VLEN / 16), a3 = LD (leading dimension in
elements)
vsetvli t0, a2, e8, m1, ta, ma
vmtl.v v1, (a0), a3          # tile-load scale_A
vmtl.v v2, (a1), a3          # tile-load scale_B
vsetvli t0, a2, e8, mf2, ta, ma
vzip.vv v0, v1, v2           # pair into 16-bit layout
```

Table 82. Scale-packing instruction count comparison

Option	Data instructions	vsetvli calls	Total
1 (vmtl.v pre-paired)	1	1	2
2 (vle16.v pre-paired)	1	1	2
3 (Base V)	5	2	7
4 (Zvbb)	4	2	6
5 (Zvzip)	3	1	4
6 (vmtl.v + vzip.vv)	3	2	5

36.7.3.6. Microscaling subextensions

Microscaling subextensions that add E8M0-scaled variants of the narrow input formats (OFP4, OFP8) are listed separately in [Table 83](#).

36.8. Zvmtls: Extension for loading and storing vector registers interpreted as 2D matrix tiles

The `Zvmtls` extension provides instructions that transfer 2D matrix tiles between memory and the vector register file without requiring a separate repacking step. Matrix data in memory is typically stored as a large 2D array in row-major or column-major order with a leading dimension that may exceed the tile width. This allows a tile to address a sub-matrix within a larger allocation. The tile load and store instructions bridge the gap between this strided 2D memory layout and the flat vector register representation used by the multiply-accumulate instructions.

36.8.1. Memory layout

Tile instructions interpret memory as a 2D array of SEW-bit elements. `rs1` holds the base address of the tile; `rs2` holds the *leading dimension* `LD`, i.e., the stride between the starts of consecutive segments, measured in elements. For a row-major matrix, `LD` equals the number of columns in the full matrix; for a column-major matrix, `LD` equals the number of rows. `LD` may be larger than the tile dimension, enabling access to a sub-tile of a larger matrix without copying.

The byte address of the element at offset `d` along the stride direction and offset `k` within a segment is:

$$\text{address}(d, k) = \text{rs1} + (d \times \text{LD} + k) \times (\text{SEW} \div 8)$$

36.8.2. Tile geometry for loads and stores

The tile dimensions are fully determined by the current **vtype** settings (SEW, VL, LMUL, and λ), consistent with the operand geometry of the multiply-accumulate instructions. The tile load and store instructions make use of the following parameters from the CSRs:

- SEW — element width
- VL — active vector length (must be a multiple of λ)
- LMUL — vector length multiplier
- λ — selected lambda, read from **lambda[2:0]** in **vtype**

When loading A or B input tiles, **vmtl.v** and **vmttl.v** shall be used with SEW equal to the element width of the C accumulator tile. Figure 14 illustrates the memory to VR load for both row-major and column-major order for a tile with LMUL=1. Physically both transfers are identical: they move contiguous segments of length $\text{linesize} = \lambda \times \text{LMUL}$ with a stride of LD between them. The tile load/store instructions interpret the memory layout according to the specified leading dimension, but the resulting data layout in the VR is the same regardless of whether the source/destination matrix is stored in row-major or column-major order.

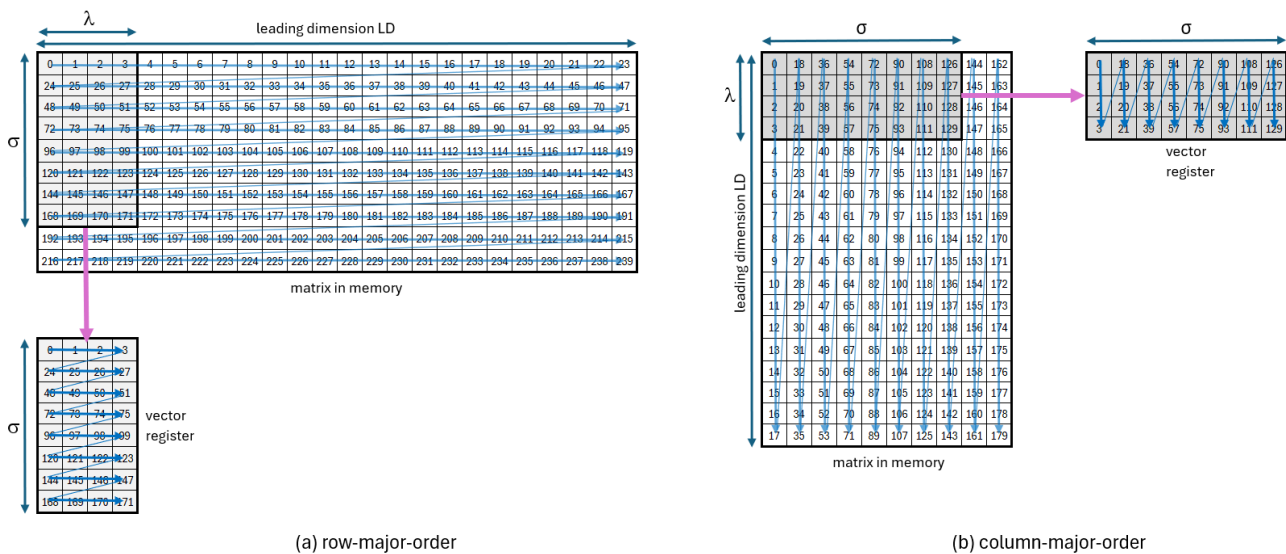


Figure 14. Loading a matrix tile from memory for LMUL=1. The matrix is layed out linearly in memory, the leading dimension LD specifies its row size (a) or column size (b). Element indices represent the offset of the elements in memory. Blue arrows indicate the data ordering in memory/VR.

If (rs2) = 0, then the leading dimension LD is set to the *natural dimension* of $\lambda \times \text{LMUL}$. That is, the memory layout, with elements contiguous to each other, matches the layout of the register group being loaded/stored.



It is expected that implementations will optimize the case of **rs2** = **x0** to load/store a vector register group with contiguous VL elements in memory.

Efficient implementation of tile loads with **rs2** = **x0** are essential to high-performance linear algebra kernels.

When using a vector length lower than the maximum for the data width, the data is loaded such that it fills

full rows in an A tile or columns in a B^T tile. The impact is sketched in Figure 15.

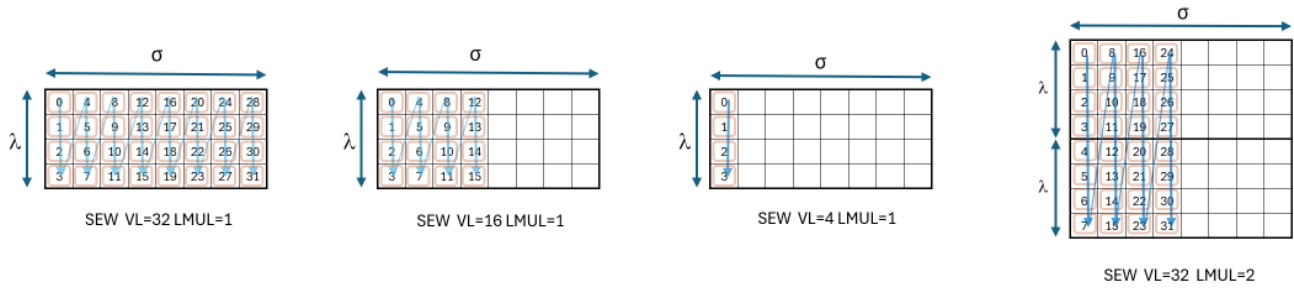


Figure 15. Examples of element distribution for full (left) and reduced vector length matrix tile load / store operations.

36.8.3. Instructions

All tile load/store instructions support two optional operands that follow **rs2** in the assembly syntax:

- An *immediate lambda* **L λ** (one of **L1**, **L2**, **L4**, **L8**, **L16**, **L32**, **L64**), which overrides **lambda[2:0]** from **vtype** for the duration of that instruction. When omitted, the dynamic lambda from **vtype** is used.
- A *mask specifier* **v0.t**, which enables element-level masking using **v0** as the mask register. When omitted, all elements are unmasked.

When both are present, the immediate lambda precedes the mask: **vmtts.v v4, (a0), a1, L64, v0.t**.

Tile load and store instructions follow the standard vector element-status semantics:

- *Active elements* (body, mask enabled): loads fetch from memory and write **vd**; stores read **vs3** and write to memory. Active elements may raise memory exceptions.
- *Inactive elements* (body, mask disabled): loads never raise exceptions and do not update the destination register unless **vtype.vma=1** (mask-agnostic), in which case those elements may be overwritten with 1s. Stores do not write to memory and do not raise exceptions.
- *Tail elements* (index $\geq$ VL): loads never raise exceptions and do not update the destination register unless **vtype.vta=1** (tail-agnostic), in which case those elements may be overwritten with 1s. Stores do not write to memory and do not raise exceptions.
- *Prestart elements* (index $<$ **vstart**): neither the destination register nor memory is updated; no exceptions are raised.

Mnemonic	Description
vmtl.v vd, (rs1), rs2 [, Lλ] [, vm]	Order-preserving tile load
vmts.v vs3, (rs1), rs2 [, Lλ] [, vm]	Order-preserving tile store

36.8.3.1. vmtl.v — Order-Preserving Tile Load

```
vmtl.v vd, (rs1), rs2 [, L $\lambda$ ] [, vm]
```

Loads a 2D matrix tile from memory into the vector register group starting at **vd**. **rs1** holds the matrix base address; **rs2** holds the leading dimension **LD** in elements.

Let $linesize = \lambda \times LMUL$. For each element index i in the body $[vstart, VL)$ where the mask is enabled:

$$VD[i] = M[rs1 + (SEW \div 8) \times ((i / linesize) \times LD + (i \% linesize))]$$

This instruction is the correct choice when A is stored in row-major order or when B is stored in column-major order: in both cases the memory layout consists of $linesize$ -element contiguous segments with a stride of LD elements between them.

36.8.3.2. vmts.v — Order-Preserving Tile Store

```
vmts.v vs3, (rs1), rs2 [, Lλ] [, vm]
```

Stores the 2D matrix tile held in the vector register group starting at **vs3** to memory. **rs1** holds the matrix base address; **rs2** holds the leading dimension LD in elements.

Let $linesize = \lambda \times LMUL$. For each element index i in the body $[vstart, VL)$ where the mask is enabled:

$$M[rs1 + (SEW \div 8) \times ((i / linesize) \times LD + (i \% linesize))] = VS[i]$$



Order preserving tile load/store with $LMUL > 1$ offers optimization opportunities. While *vmtl/vmts* are very similar to vector constant-stride segment operations, the segment sizes are potentially larger. Matching cache line size with $\lambda \times LMUL \times SEW$ allows for full cacheline transfers.

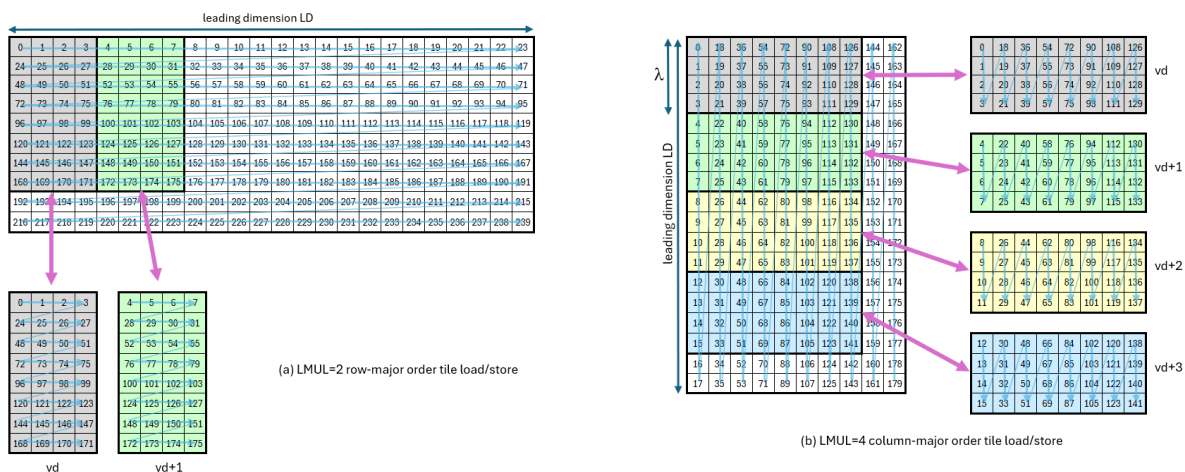


Figure 16. Order preserving tile load/store with $LMUL > 1$ for row-major (a) and column-major ordering in memory.

36.9. Zvmmtts: Extension for loading and storing vector registers interpreted as transposed 2D matrix tiles

The *Zvmmtts* extension provides instructions that transfer transposed 2D matrix tiles between memory and the vector register file without requiring a separate repacking step. Memory layout and tile geometry for *Zvmmtts* follows those established for the *Zvmtls* extension.

36.9.1. Instructions

All tile load/store instructions support two optional operands that follow **rs2** in the assembly syntax:

- An *immediate lambda* **Lλ** (one of **L1**, **L2**, **L4**, **L8**, **L16**, **L32**, **L64**), which overrides **lambda[2:0]** from **vtype** for the duration of that instruction. When omitted, the dynamic lambda from **vtype** is used.
- A *mask* specifier **v0.t**, which enables element-level masking using **v0** as the mask register. When omitted, all elements are unmasked.

When both are present, the immediate lambda precedes the mask: **vmtts.v v4, (a0), a1, L64, v0.t**.

Tile load and store instructions follow the standard vector element-status semantics:

- *Active* elements (body, mask enabled): loads fetch from memory and write **vd**; stores read **vs3** and write to memory. Active elements may raise memory exceptions.
- *Inactive* elements (body, mask disabled): loads never raise exceptions and do not update the destination register unless **vtype.vma=1** (mask-agnostic), in which case those elements may be overwritten with 1s. Stores do not write to memory and do not raise exceptions.
- *Tail* elements (index $\geq$ VL): loads never raise exceptions and do not update the destination register unless **vtype.vta=1** (tail-agnostic), in which case those elements may be overwritten with 1s. Stores do not write to memory and do not raise exceptions.
- *Prestart* elements (index $<$ **vstart**): neither the destination register nor memory is updated; no exceptions are raised.

Mnemonic	Description
vmttl.v vd, (rs1), rs2 [, Lλ] [, vm]	Transposing tile load
vmtts.v vs3, (rs1), rs2 [, Lλ] [, vm]	Transposing tile store

36.9.1.1. vmttl.v — Transposing Tile Load

```
vmttl.v vd, (rs1), rs2 [, Lλ] [, vm]
```

Loads a 2D matrix tile from memory and transposes it into the vector register layout required by the matrix multiplier. This instruction is used when a B tile is stored in row-major order, or when an A or C tile is stored in column-major order in memory. **rs1** holds the matrix base address; **rs2** holds the leading dimension **LD** in elements.

Let $linesize = \lambda \times LMUL$. For each element index **i** in the body [**vstart**, VL) where the mask is enabled:

```
VD[i] = M[rs1 + (SEW ÷ 8) × ((i % linesize) × LD + (i / linesize))]
```

36.9.1.2. vmtts.v — Transposing Tile Store

```
vmtts.v vs3, (rs1), rs2 [, Lλ] [, vm]
```


Stores a 2D matrix tile from vector registers to memory, applying the inverse transposition of `vmttl.v`. `rs1` holds the matrix base address; `rs2` holds the leading dimension `LD` in elements.

Let $linesize = \lambda \times LMUL$. For each element index `i` in the body `[vstart, VL)` where the mask is enabled:

$$M[rs1 + (SEW \div 8) \times ((i \% linesize) \times LD + (i / linesize))] = VS[i]$$



Transposing loads and stores should be performed with SEW set to the width of the actual data that is being loaded and transposed. Sub-byte elements (eg. OFP4) cannot be transposed directly.

36.9.2. Tile geometry for multiply-accumulate

The tile dimensions are fully determined by the current vector configuration:

M_tile	= $VLEN / (SEW \times \lambda)$	(rows of A and C per computation)
K_eff	= $\lambda \times W \times LMUL$	(shared K-dimension step)
N_tile	= $VL / (\lambda \times LMUL)$	(active columns of B and C; set via <code>vsetvl</code>)
N_tile_max	= M_tile	(maximum columns when $VL = VL_{max}$)
MUL_C	= $VLEN / (SEW \times \lambda^2)$	(C accumulator register group size)

M_tile is fixed by the hardware ($VLEN$) and the chosen SEW and λ ; it cannot be changed by VL . **N_tile** is controlled by the programmer through VL ; use `vsetvl` to select the largest **N_tile** that fits the remaining columns. VL must be a multiple of $\lambda \times LMUL$, so only full columns can be selected.

Because the multiply-accumulate instructions always read all **M_tile** rows of A independent of VL , loading an A tile requires $VL = M_tile \times K_eff$ (the natural maximum VL for the chosen $LMUL$). The B load and computation use the narrower $VL = K_eff \times N_tile$.

36.9.3. C tile tail policy

When $N_tile < N_tile_max$, the elements $C_{i,j}$ for $0 \leq i < M_tile$ and $N_tile \leq j < N_tile_max$ are *C tile tail elements*. Multiply-accumulate instructions never read or write C tile tail elements.

- If **vta** = 0 (tail-undisturbed): tail elements retain their pre-instruction values. This is achieved by write-skip — the instruction simply does not access those element positions. Implementations are therefore not required to read the tail portion of the C register group (i.e., all registers that fall entirely into the tail).
- If **vta** = 1 (tail-agnostic): tail elements may be written with any value (including all-ones), or left unchanged. Implementations are likewise not required to read the tail elements before deciding what to write.



This differs from the tail policy for ordinary vector instructions, where tail-undisturbed is typically implemented via read-merge: the implementation reads the old destination register contents, updates active positions, and writes the merged result back. For IME multiply-accumulate instructions, undisturbed behaviour requires no read — the instruction leaves tail element positions untouched. This permits hardware implementations (including outer-product engines and register-renaming machines) to process only the N_tile active columns of the C tile without referencing the tail portion of the C register group (i.e., all registers that fall entirely into the tail).

36.9.4. Memory layout and instruction selection

Matrix / memory order	Load instruction	Store instruction
A, row-major	vm_{tl}.v (order-preserving)	—
A, column-major	vm_{ttl}.v (transposing)	vm_{tts}.v
B, column-major	vm_{tl}.v (order-preserving)	—
B, row-major	vm_{ttl}.v (transposing)	—
C, column-major	vm_{tl}.v (if loaded from memory)	vm_{ts}.v

Matrix / memory order	Load instruction	Store instruction
C, row-major	<code>vmttl.v</code> (transposing)	<code>vmmts.v</code>

36.9.5. Example: single-precision floating-point GEMM

The following pseudocode implements $C \leftarrow C + A \times B$ for single-precision floating-point data, tiled over all three matrix dimensions. The configuration uses SEW=32, LMUL=1, and $\lambda=4$, giving $K_{\text{eff}} = 4$ and $M_{\text{tile}} = \text{VLEN}/128$. A is stored row-major; B and C are stored column-major.

```
// C (M×N, col-major, stride ldc) += A (M×K, row-major, stride lda)
//                                     × B (K×N, col-major, stride ldb)
// Pre-condition: vtype configured for SEW=32, LMUL=1, lambda=4
// Pre-condition: K is a multiple of K_eff (= lambda × LMUL = 4)

const int K_eff = 4;                // lambda × LMUL
const int M_tile = VLEN / (32 * 4); // VLEN / (SEW × lambda)

for (j = 0; j < N; j += N_tile) {
    // Choose N_tile: how many B/C columns to handle this iteration.
    // vsetvl rounds down to a multiple of K_eff automatically.
    vl = vsetvl_e32m1(K_eff * (N - j));
    N_tile = vl / K_eff;

    for (i = 0; i < M; i += M_tile) {

        // Zero the C tile accumulator (MUL_C registers starting at v8).
        vsetvl(K_eff * M_tile); // VL = vl_max, needed to touch all elements
        vmv.v.i v8, 0;

        // Inner K loop: accumulate A sub-columns × B sub-rows into C tile.
        for (k = 0; k < K; k += K_eff) {

            // Load A[i:i+M_tile][k:k+K_eff]: M_tile rows, K_eff cols, row-
major.
            // Must use VL = M_tile × K_eff so all rows are written to v0.
            vsetvl(K_eff * M_tile); // VL = vl_max
            vmttl.v v0, &A[i*lda + k], lda; // LD = lda (A row stride)

            // Load B[k:k+K_eff][j:j+N_tile]: K_eff rows, N_tile cols, col-
major.
            // Then accumulate into C tile.
            vsetvl(vl); // VL = K_eff × N_tile
            vmttl.v v4, &B[j*ldb + k], ldb; // LD = ldb (B col stride =
K_total)
            vfmmacc.vv v8, v0, v4; // C[i..][j..] += A_tile × B_tile
        }

        // Store C[i:i+M_tile][j:j+N_tile]: M_tile rows, N_tile cols, col-
major.
    }
}
```

```

        vsetvl(vl);
        vmts.v v8, &C[j*ldc + i], ldc;          // LD = ldc (C col stride =
M_total)
    }
}

```

Key observations:

- The A tile must be loaded with $VL = M_tile \times \lambda \times LMUL$ (the maximum VL for the chosen LMUL) because `vfmacc.vv` always reads all `M_tile` rows of A regardless of the VL set for the computation. A separate `vsetvl` is therefore required before each `vmtl.v` for A.
- The B load and the multiply-accumulate instruction share $VL = \lambda \times LMUL \times N_tile$, which restricts computation to the current column block.
- The A and B tile loads must be performed with the target SEW (element width of the C tile).
- For integer GEMM, substitute `vmmacc.vv` for `vfmacc.vv` and set `altfmt_A` and `altfmt_B` in `vtype` to select the desired operand signedness. The load instructions are unchanged.
- For widening variants ($W=2$ or $W=4$), use `vfwmmacc.vv/vfqwmmacc.vv` or `vwmacc.vv/vqwmmacc.vv`. The narrower A and B element widths are encoded in the multiply-accumulate instruction; the tile load instructions require no modification.

36.10. Software considerations

36.10.1. C-Language Intrinsics

The IME C-language intrinsics provide portable access to the IME instructions without requiring hand-written assembly. The naming convention and type system extend the RISC-V V Intrinsics API (RISC-V International, *RISC-V V Intrinsics API Specification*):

- Function names take the form `_riscv{mnemonic}_{type-suffix}`.
- The *type-suffix* encodes the **accumulator C** (for multiply-accumulate), **destination** (for loads), or **source-vector** (for stores) element type and register-group multiplier, using the same codes as the RISC-V V intrinsics: `i8m1` $\rightarrow$ `vint8m1_t`, `f32m2` $\rightarrow$ `vfloat32m2_t`, `u16m4` $\rightarrow$ `vuint16m4_t`, etc. For multiply-accumulate intrinsics the register-group multiplier encoded in the type-suffix is `MUL_C` ($= \text{VLEN} \div (\text{SEW} \times \lambda^2)$), which is determined by the implementation's `VLEN` and is *independent of LMUL*.
- When `MUL_C` $\neq$ `LMUL` for a multiply-accumulate intrinsic, a `_lm{N}` qualifier ($N \in \{1, 2, 4, 8\}$) is appended to the intrinsic name to specify `LMUL`. The `_lm1` qualifier may be omitted; `LMUL=1` is the default when no qualifier is present. The `vs1` and `vs2` argument types directly reflect `LMUL`: `vx{EEW}m{LMUL}_t`.
- Every intrinsic accepts a final `size_t vl` argument equal to the value returned by the immediately preceding `vsetvl` call.
- All intrinsics are implicitly barrier-like with respect to the vector register file: the compiler must not reorder them across other vector operations that touch the same register state.
- When the input element type is non-ISO (`BFloat16`, `OFP8`, `OFP4`, or `Int4`), the input-type name and register-group multiplier are appended as a suffix (`_{type}m{LMUL}`) to the intrinsic name, even for the default `altfmt=0` encoding. When the input type is a standard IEEE floating-point format (`FP16`, `FP32`, `FP64`) or a standard-width integer (`Int8`–`Int64`), no input-type suffix is needed for the default encoding.
- When `altfmt_A` $\neq$ `altfmt_B` (mixed input formats), both input-type suffixes appear separated by an underscore, with the `A` suffix first: `_riscv{mnemonic}_vv_{accum}_{inputA}_{inputB}`. When `altfmt_A` $=$ `altfmt_B`, a single input-type suffix suffices.
- The canonical suffix ordering for the long-form name is: `{type-suffix}[{inputA}[{inputB}]][_su|_us][_lm{N}][_L{N}]` where each bracketed component is omitted when not applicable.

36.10.1.1. Overloaded short forms (GCC and Clang)

In addition to the canonical long-form names described above, each intrinsic is available under the shorter name `riscv_{mnemonic}` with compiler-managed overload resolution: GCC resolves calls through its `resolve_overloaded_builtin` mechanism; Clang uses `attribute__overloadable`; C++ compilers use standard function overloading. In these short forms the compiler selects the correct instruction variant solely from the types of `vd`, `vs1`, and `vs2`; no `_lm{N}` qualifier, no input-type suffix, and no `_su` / `_us` sign suffix is needed in user code.

The types of the arguments fully determine every variant:

- `vd` type $\rightarrow$ accumulator `SEW` and `MUL_C`.
- `vs1` / `vs2` types $\rightarrow$ input `EEW` and `LMUL`; for integer operations, signed vs. unsigned; for floating-point operations, the FP format (e.g., `FP16` vs. `BFloat16` for `altfmt`).

```
/* Long forms – always portable, compiler-independent: */
```

```

vint32m2_t __riscv_vmmacc_vv_i32m2    (vint32m2_t vd, vint32m1_t vs1,
vint32m1_t vs2, size_t vl);
vint32m2_t __riscv_vmmacc_vv_i32m2_lm2(vint32m2_t vd, vint32m2_t vs1,
vint32m2_t vs2, size_t vl);

/* Equivalent short forms – overload resolution picks the right variant: */
vint32m2_t __riscv_vmmacc_vv(vint32m2_t vd, vint32m1_t vs1, vint32m1_t vs2,
size_t vl) __attribute__((overloadable));
vint32m2_t __riscv_vmmacc_vv(vint32m2_t vd, vint32m2_t vs1, vint32m2_t vs2,
size_t vl) __attribute__((overloadable));

/* FP format (altfmt) resolved by vs1/vs2 type – no suffix needed: */
vfloat32m2_t __riscv_vfwmmacc_vv(vfloat32m2_t vd, vfloat16m1_t vs1,
vfloat16m1_t vs2, size_t vl) __attribute__((overloadable));
vfloat32m2_t __riscv_vfwmmacc_vv(vfloat32m2_t vd, vfloat16m1_t vs1,
vfloat16m1_t vs2, size_t vl) __attribute__((overloadable));

/* Integer sign combination resolved by vs1/vs2 types – no _su/_us suffix: */
vint32m2_t __riscv_vwmacc_vv(vint32m2_t vd, vint16m1_t vs1, vint16m1_t vs2,
size_t vl) __attribute__((overloadable));
vint32m2_t __riscv_vwmacc_vv(vint32m2_t vd, vint16m1_t vs1, vint16m1_t vs2,
size_t vl) __attribute__((overloadable));

```



The mechanism used for overload resolution in C is compiler-specific (resolve_overloaded_builtin in GCC, attribute(overloadable) in Clang). The resulting user-visible API is identical across compilers; only the internal implementation differs.

36.10.1.2. Tile load and store (Zvmtls) and transposed tile load and store (Zvmttls)

The load intrinsics accept a base pointer, a leading-dimension `ld` in elements, and a vector-length `vl`, returning the loaded vector type. The store intrinsics accept a base pointer, `ld`, the source vector, and `vl`, and return `void`. For tile loads and stores the type-suffix encodes LMUL—the register-group multiplier of the tile operand. The table below lists representative `m1` prototypes; replace `m1` with `m2`, `m4`, `m8`, `mf2`, `mf4`, or `mf8` as needed, and `i` with `u` for unsigned integer variants.

```

/* vmtl.v – order-preserving tile load */
vint8m1_t    __riscv_vmtl_v_i8m1  (const int8_t   *base, size_t ld, size_t vl);
vint16m1_t   __riscv_vmtl_v_i16m1 (const int16_t  *base, size_t ld, size_t vl);
vint32m1_t   __riscv_vmtl_v_i32m1 (const int32_t  *base, size_t ld, size_t vl);
vint64m1_t   __riscv_vmtl_v_i64m1 (const int64_t  *base, size_t ld, size_t vl);
vfloat16m1_t __riscv_vmtl_v_f16m1 (const _Float16 *base, size_t ld, size_t vl);
vfloat32m1_t __riscv_vmtl_v_f32m1 (const float    *base, size_t ld, size_t vl);
vfloat64m1_t __riscv_vmtl_v_f64m1 (const double   *base, size_t ld, size_t vl);

/* vmts.v – order-preserving tile store */
void __riscv_vmts_v_i8m1  (int8_t   *base, size_t ld, vint8m1_t  vs3, size_t vl);
void __riscv_vmts_v_i16m1 (int16_t  *base, size_t ld, vint16m1_t vs3, size_t vl);

```

```

void __riscv_vmts_v_i32m1 (int32_t *base, size_t ld, vint32m1_t vs3, size_t vl);
void __riscv_vmts_v_i64m1 (int64_t *base, size_t ld, vint64m1_t vs3, size_t vl);
void __riscv_vmts_v_f16m1 (_Float16 *base, size_t ld, vfloat16m1_t vs3, size_t vl);
void __riscv_vmts_v_f32m1 (float *base, size_t ld, vfloat32m1_t vs3, size_t vl);
void __riscv_vmts_v_f64m1 (double *base, size_t ld, vfloat64m1_t vs3, size_t vl);

/* vmttl.v - transposing tile load */
vint8m1_t __riscv_vmttl_v_i8m1 (const int8_t *base, size_t ld, size_t vl);
vint16m1_t __riscv_vmttl_v_i16m1 (const int16_t *base, size_t ld, size_t vl);
vint32m1_t __riscv_vmttl_v_i32m1 (const int32_t *base, size_t ld, size_t vl);
vint64m1_t __riscv_vmttl_v_i64m1 (const int64_t *base, size_t ld, size_t vl);
vfloat16m1_t __riscv_vmttl_v_f16m1 (const _Float16 *base, size_t ld, size_t vl);
vfloat32m1_t __riscv_vmttl_v_f32m1 (const float *base, size_t ld, size_t vl);
vfloat64m1_t __riscv_vmttl_v_f64m1 (const double *base, size_t ld, size_t vl);

/* vmtts.v - transposing tile store */
void __riscv_vmtts_v_i8m1 (int8_t *base, size_t ld, vint8m1_t vs3, size_t vl);
void __riscv_vmtts_v_i16m1 (int16_t *base, size_t ld, vint16m1_t vs3, size_t vl);
void __riscv_vmtts_v_i32m1 (int32_t *base, size_t ld, vint32m1_t vs3, size_t vl);
void __riscv_vmtts_v_i64m1 (int64_t *base, size_t ld, vint64m1_t vs3, size_t vl);
void __riscv_vmtts_v_f16m1 (_Float16 *base, size_t ld, vfloat16m1_t vs3, size_t vl);
void __riscv_vmtts_v_f32m1 (float *base, size_t ld, vfloat32m1_t vs3, size_t vl);
void __riscv_vmtts_v_f64m1 (double *base, size_t ld, vfloat64m1_t vs3, size_t vl);

```

When an immediate lambda override is required, a `_L{N}` variant encodes the lambda value ($N \in \{1, 2, 4, 8, 16, 32, 64\}$) directly in the intrinsic name. Because the lambda value maps to a 3-bit instruction-immediate field, the compiler can statically verify legality; no runtime argument is needed.

```

/* vmttl.v with lambda=4: */
vint32m1_t __riscv_vmttl_v_i32m1_L4 (const int32_t *base, size_t ld, size_t vl);

```

```

vl);
/* vmts.v with lambda=4: */
void      __riscv_vmts_v_i32m1_L4 (int32_t *base, size_t ld, vint32m1_t
vs3, size_t vl);
/* vmttl.v with lambda=4: */
vint32m1_t __riscv_vmttl_v_i32m1_L4 (const int32_t *base, size_t ld, size_t
vl);
/* vmtts.v with lambda=4: */
void      __riscv_vmtts_v_i32m1_L4 (int32_t *base, size_t ld, vint32m1_t
vs3, size_t vl);

```

36.10.1.3. Integer matrix multiply-accumulate (Zvmm)

The accumulator register group C uses $MUL_C = VLEN \div (SEW \times \lambda^2)$ registers. MUL_C is determined by the implementation's $VLEN$ and is independent of $LMUL$. The type-suffix in the intrinsic name always encodes the accumulator C type (including its MUL_C); A and B use **vs1** and **vs2** argument types that reflect $LMUL$ directly. The tables below list representative prototypes for $VLEN=256$ and $\lambda=2$ (giving $MUL_C = VLEN \div (SEW \times 4)$); all valid (MUL_C , $LMUL$) combinations are available with the appropriate `_lm{N}` qualifier.

For `vmacc.vv` (no widening, $W=1$), accumulator and both input tiles use SEW :

```

/* SEW=8, VLEN=256, λ=2: MUL_C=8. LMUL=1 (default): */
vint8m8_t __riscv_vmmacc_vv_i8m8 (vint8m8_t vd, vint8m1_t vs1, vint8m1_t
vs2, size_t vl);
/* SEW=16, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vint16m4_t __riscv_vmmacc_vv_i16m4 (vint16m4_t vd, vint16m1_t vs1, vint16m1_t
vs2, size_t vl);
/* SEW=32, VLEN=256, λ=2: MUL_C=2. LMUL=1 (default): */
vint32m2_t __riscv_vmmacc_vv_i32m2 (vint32m2_t vd, vint32m1_t vs1, vint32m1_t
vs2, size_t vl);
/* Same MUL_C=2, LMUL=2 (_lm2 qualifier): */
vint32m2_t __riscv_vmmacc_vv_i32m2_lm2(vint32m2_t vd, vint32m2_t vs1,
vint32m2_t vs2, size_t vl);
/* SEW=64, VLEN=256, λ=2: MUL_C=1. LMUL=1 (default, MUL_C=LMUL): */
vint64m1_t __riscv_vmmacc_vv_i64m1 (vint64m1_t vd, vint64m1_t vs1, vint64m1_t
vs2, size_t vl);
/* Unsigned variants: __riscv_vmmacc_vv_u{sew}m{MUL_C}[_lm{N}], accepting vuint
types. */

```

For `vmmacc.vv` ($W=2$), A and B use $SEW/2$ elements; the accumulator uses SEW . The type-suffix on `vd` determines the accumulator $\odot$ SEW and MUL_C :

```

/* SEW=16 accum / EEW=8 inputs, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vint16m4_t __riscv_vmmacc_vv_i16m4 (vint16m4_t vd, vint8m1_t vs1, vint8m1_t
vs2, size_t vl);
/* SEW=32 accum / EEW=16 inputs, VLEN=256, λ=2: MUL_C=2. LMUL=1 (default): */
vint32m2_t __riscv_vmmacc_vv_i32m2 (vint32m2_t vd, vint16m1_t vs1,

```



```
vint16m1_t vs2, size_t vl);
/* SEW=64 accum / EEW=32 inputs, VLEN=256, λ=2: MUL_C=1. LMUL=1 (default): */
vint64m1_t __riscv_vwmmacc_vv_i64m1 (vint64m1_t vd, vint32m1_t vs1,
vint32m1_t vs2, size_t vl);
```

For `vqwmacc.vv` ($W=4$), A and B use SEW/4 elements:

```
/* SEW=32 accum / EEW=8 inputs, VLEN=256, λ=2: MUL_C=2. LMUL=1 (default): */
vint32m2_t __riscv_vqwmacc_vv_i32m2 (vint32m2_t vd, vint8m1_t vs1, vint8m1_t
vs2, size_t vl);
/* SEW=64 accum / EEW=16 inputs, VLEN=256, λ=2: MUL_C=1. LMUL=1 (default): */
vint64m1_t __riscv_vqwmacc_vv_i64m1 (vint64m1_t vd, vint16m1_t vs1,
vint16m1_t vs2, size_t vl);
```

The `vint4m{N}_t` and `vuint4m{N}_t` vector types are implementation-defined extensions for 4-bit integer elements, analogous to the OFP4 vector types for 4-bit floating-point.

For `vwmacc.vv` ($W=2$) with SEW=8, the inputs are 4-bit integers (`Zvvi4i8mm`):

```
/* Zvvi4i8mm: Int4→Int8, SEW=8, VLEN=256, λ=2: MUL_C=8. LMUL=1 (default): */
vint8m8_t __riscv_vwmacc_vv_i8m8_i4m1 (vint8m8_t vd,
vint4m1_t vs1, vint4m1_t vs2, size_t
vl);
```

For `vqwmacc.vv` ($W=4$) with SEW=16, the inputs are 4-bit integers (`Zvvi4i16mm`):

```
/* Zvvi4i16mm: Int4→Int16, SEW=16, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default):
*/
vint16m4_t __riscv_vqwmacc_vv_i16m4_i4m1(vint16m4_t vd,
vint4m1_t vs1, vint4m1_t vs2,
size_t vl);
```

The following subextensions require an effective widening ratio of 8 ($W=8$) and are served by the `v8wmacc.vv` (integer) and `vf8wmacc.vv` (floating-point) instructions:

- Integer: `Zvvi4i32mm` ($\text{Int4} \rightarrow \text{Int32}$), `Zvvi8i64mm` ($\text{Int8} \rightarrow \text{Int64}$)
- Base FP: `Zvvofp4fp32mm` ($\text{OFP4} \rightarrow \text{FP32}$), `Zvvofp8fp64mm` ($\text{OFP8} \rightarrow \text{FP64}$)
- Microscaling: `Zvvxofp4fp32mm` / `Zvvxnofp4fp32mm` ($\text{MXFP4} \rightarrow \text{FP32}$), `Zvvxofp8fp64mm` / `Zvvxnofp8fp64mm` ($\text{MXFP8} \rightarrow \text{FP64}$), `Zvvxi8fp64mm` / `Zvvxni8fp64mm` ($\text{MXINT8} \rightarrow \text{FP64}$)

The `altfmt_A` and `altfmt_B` fields in `vtype` select the signedness of A and B independently. When mixed-sign multiplication is needed (signed A $\times$ unsigned B, or vice versa), use the `_su` (signed $\times$ unsigned) or `_us` (unsigned $\times$ signed) suffix on the intrinsic name:

```
/* vwmacc.vv signed A  $\times$  unsigned B; SEW=32, VLEN=256, λ=2: MUL_C=2, LMUL=1 */
vint32m2_t __riscv_vwmacc_vv_i32m2_su(vint32m2_t vd, vint16m1_t vs1,
```

```

                                vuint16m1_t vs2, size_t vl);
/* vwmacc.vv unsigned A × signed B; SEW=32, VLEN=256, λ=2: MUL_C=2, LMUL=1 */
vint32m2_t __riscv_vwmacc_vv_i32m2_us(vint32m2_t vd, vuint16m1_t vs1,
                                vint16m1_t vs2, size_t vl);

```

36.10.1.4. Floating-point matrix multiply-accumulate (Zvvfmm)

The OFP8 vector types `vfloat8e4m3mX_t` (E4M3) and `vfloat8e5m2mX_t` (E5M2), and the OFP4 vector type `vfloat4e2m1mX_t` (E2M1), are implementation-defined extensions following the V intrinsics naming convention for non-ISO types, analogous to `vbfloat16mX_t` for BFloat16. The corresponding scalar types (`_Float8E4M3`, `_Float8E5M2`, `_Float4E2M1`) are not part of ISO C; once standardised, headers will alias these to the ISO names.

For `vfmacc.vv` (no widening, $W=1$), accumulator and input tiles share SEW:

```

/* OFP8 E4M3, VLEN=256, λ=2: MUL_C=8. LMUL=1 (default): */
vfloat8e4m3m8_t __riscv_vfmacc_vv_f8e4m3m8(vfloat8e4m3m8_t vd,
                                vfloat8e4m3m1_t vs1,
                                vfloat8e4m3m1_t vs2, size_t vl);
/* OFP8 E5M2, VLEN=256, λ=2: MUL_C=8. LMUL=1 (default): */
vfloat8e5m2m8_t __riscv_vfmacc_vv_f8e5m2m8(vfloat8e5m2m8_t vd,
                                vfloat8e5m2m1_t vs1,
                                vfloat8e5m2m1_t vs2, size_t vl);
/* FP16, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vfloat16m4_t __riscv_vfmacc_vv_f16m4(vfloat16m4_t vd,
                                vfloat16m1_t vs1, vfloat16m1_t vs2,
                                size_t vl);
/* BF16, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vbfloat16m4_t __riscv_vfmacc_vv_bf16m4(vbfloat16m4_t vd,
                                vbfloat16m1_t vs1, vbfloat16m1_t vs2,
                                size_t vl);
/* FP32, VLEN=256, λ=2: MUL_C=2. LMUL=1 (default): */
vfloat32m2_t __riscv_vfmacc_vv_f32m2(vfloat32m2_t vd,
                                vfloat32m1_t vs1, vfloat32m1_t vs2,
                                size_t vl);
/* FP64, VLEN=256, λ=2: MUL_C=1. LMUL=1 (default, MUL_C=LMUL): */
vfloat64m1_t __riscv_vfmacc_vv_f64m1(vfloat64m1_t vd,
                                vfloat64m1_t vs1, vfloat64m1_t vs2,
                                size_t vl);

```

For `vfwmacc.vv` ($W=2$), A/B use SEW/2:

```

/* OFP8 E4M3 accum / OFP4 E2M1 inputs, VLEN=256, λ=2: MUL_C=8. LMUL=1
(default): */
vfloat8e4m3m8_t __riscv_vfwmacc_vv_f8e4m3m8_f4e2m1m1(
    vfloat8e4m3m8_t vd, vfloat4e2m1m1_t vs1, vfloat4e2m1m1_t vs2, size_t vl);
/* FP16 accum / OFP8 E4M3 inputs, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vfloat16m4_t __riscv_vfwmacc_vv_f16m4_f8e4m3m1(vfloat16m4_t vd,

```

```

vfloat8e4m3m1_t vs1,
vfloat8e4m3m1_t vs2, size_t vl);
/* BF16 accum / OFP8 E4M3 inputs, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vbfloat16m4_t __riscv_vfwmmacc_vv_bf16m4_f8e4m3m1(vbfloat16m4_t vd,
vfloat8e4m3m1_t vs1,
vfloat8e4m3m1_t vs2, size_t vl);
/* FP32 accum / FP16 inputs, VLEN=256, λ=2: MUL_C=2. LMUL=1 (default): */
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2(vfloat32m2_t vd,
vfloat16m1_t vs1, vfloat16m1_t vs2,
size_t vl);
/* FP64 accum / FP32 inputs, VLEN=256, λ=2: MUL_C=1. LMUL=1 (default): */
vfloat64m1_t __riscv_vfwmmacc_vv_f64m1(vfloat64m1_t vd,
vfloat32m1_t vs1, vfloat32m1_t vs2,
size_t vl);

```

For `vfqwmacc.vv` (W=4), A/B use SEW/4:

```

/* FP16 accum / OFP4 E2M1 inputs, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vfloat16m4_t __riscv_vfqwmacc_vv_f16m4_f4e2m1m1(vfloat16m4_t vd,
vfloat4e2m1m1_t vs1,
vfloat4e2m1m1_t vs2, size_t vl);
/* BF16 accum / OFP4 E2M1 inputs, VLEN=256, λ=2: MUL_C=4. LMUL=1 (default): */
vbfloat16m4_t __riscv_vfqwmacc_vv_bf16m4_f4e2m1m1(vbfloat16m4_t vd,
vfloat4e2m1m1_t vs1,
vfloat4e2m1m1_t vs2, size_t vl);
/* FP32 accum / OFP8 E4M3 inputs, VLEN=256, λ=2: MUL_C=2. LMUL=1 (default): */
vfloat32m2_t __riscv_vfqwmacc_vv_f32m2_f8e4m3m1(vfloat32m2_t vd,
vfloat8e4m3m1_t vs1,
vfloat8e4m3m1_t vs2, size_t vl);
/* FP64 accum / BF16 inputs, VLEN=256, λ=2: MUL_C=1. LMUL=1 (default): */
vfloat64m1_t __riscv_vfqwmacc_vv_f64m1_bf16m1(vfloat64m1_t vd,
vbfloat16m1_t vs1,
vbfloat16m1_t vs2, size_t vl);
/* FP64 accum / FP16 inputs, VLEN=256, λ=2: MUL_C=1. LMUL=1 (default): */
vfloat64m1_t __riscv_vfqwmacc_vv_f64m1(vfloat64m1_t vd,
vfloat16m1_t vs1, vfloat16m1_t vs2,
size_t vl);

```

The `altfmt_A` and `altfmt_B` fields in `vtype` select the floating-point format of the A and B input tiles. When a specific alternative format is required, the `vs1/vs2` input element-type name is appended to the intrinsic name in place of the default input type; no separate `_alt` suffix is used. The following examples show `altfmt_A=altfmt_B=1` variants for selected combinations:

```

/* vfwmmacc.vv FP32 accum / BF16 inputs (altfmt_A=B=1 for 16-bit inputs): */
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2_bf16m1(vfloat32m2_t vd,
vbfloat16m1_t vs1, vbfloat16m1_t
vs2, size_t vl);
/* vfwmmacc.vv FP16 accum / OFP8 E5M2 inputs (altfmt_A=B=1 for 8-bit inputs):

```

```

*/
vfloat16m4_t __riscv_vfwmmacc_vv_f16m4_f8e5m2m1(vfloat16m4_t vd,
                                                  vfloat8e5m2m1_t vs1,
                                                  vfloat8e5m2m1_t vs2, size_t vl);
/* vfwmmacc.vv FP32 accum / OFP8 E5M2 inputs (altfmt_A=B=1 for 8-bit inputs):
*/
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2_f8e5m2m1(vfloat32m2_t vd,
                                                  vfloat8e5m2m1_t vs1,
                                                  vfloat8e5m2m1_t vs2, size_t vl);

```

When `altfmt_A` $\neq$ `altfmt_B`, the two input type suffixes differ:

```

/* Mixed OFP8: E4M3  $\times$  E5M2  $\rightarrow$  FP16 via vfwmmacc.vv (altfmt_A=0, altfmt_B=1): */
vfloat16m4_t __riscv_vfwmmacc_vv_f16m4_f8e4m3m1_f8e5m2m1(vfloat16m4_t vd,
                                                           vfloat8e4m3m1_t
vs1, vfloat8e5m2m1_t vs2, size_t vl);
/* Mixed 16-bit: FP16  $\times$  BF16  $\rightarrow$  FP32 via vfwmmacc.vv (altfmt_A=0, altfmt_B=1):
*/
vfloat32m2_t __riscv_vfwmmacc_vv_f32m2_f16m1_bf16m1(vfloat32m2_t vd,
                                                       vfloat16m1_t vs1,
vbf16m1_t vs2, size_t vl);
/* Overloaded short forms – compiler resolves from vs1/vs2 types: */
vfloat16m4_t __riscv_vfwmmacc_vv(vfloat16m4_t vd,
                                vfloat8e4m3m1_t vs1, vfloat8e5m2m1_t vs2,
size_t vl) __attribute__((overloadable));
vfloat32m2_t __riscv_vfwmmacc_vv(vfloat32m2_t vd,
                                vfloat16m1_t vs1, vbf16m1_t vs2, size_t
vl) __attribute__((overloadable));

```

The `vfwmmacc.vv` and `vfqmmacc.vv` instructions use integer input tiles with a floating-point accumulator. The input tile type reflects the integer element width and signedness; the accumulator type reflects the FP format.

```

/* MXINT8  $\rightarrow$  FP16: vfwmmacc.vv, SEW=16, Int8 $\times$ Int8 (signed), VLEN=256,  $\lambda=2$  */
vfloat16m4_t __riscv_vfwmmacc_vv_f16m4_i8m1(vfloat16m4_t vd,
                                              vint8m1_t vs1, vint8m1_t vs2,
size_t vl);
/* MXINT8  $\rightarrow$  BF16: vfwmmacc.vv, SEW=16, UInt8 $\times$ Int8 (mixed signedness) */
vbf16m4_t __riscv_vfwmmacc_vv_bf16m4_u8m1_i8m1(vbf16m4_t vd,
                                                  vuint8m1_t vs1, vint8m1_t
vs2, size_t vl);
/* MXINT8  $\rightarrow$  FP32: vfqmmacc.vv, SEW=32, Int8 $\times$ Int8, VLEN=256,  $\lambda=2$  */
vfloat32m2_t __riscv_vfqmmacc_vv_f32m2_i8m1(vfloat32m2_t vd,
                                              vint8m1_t vs1, vint8m1_t vs2,
size_t vl);
/* MXINT4  $\rightarrow$  FP16: vfqmmacc.vv, SEW=16, Int4 $\times$ Int4 */
vfloat16m4_t __riscv_vfqmmacc_vv_f16m4_i4m1(vfloat16m4_t vd,
                                              vint4m1_t vs1, vint4m1_t vs2,

```

```
size_t vl);
/* MXINT4 → BF16: vfwimmac.vv, SEW=16, UInt4×UInt4 */
vbf16m4_t __riscv_vfwimmac_vv_bf16m4_u4m1(vbf16m4_t vd,
                                           vuint4m1_t vs1, vuint4m1_t
vs2, size_t vl);
```

36.10.1.5. VLEN-portable code

Because $MUL_C = VLEN / (SEW \times \lambda^2)$, the accumulator register-group multiplier varies across implementations with different values of VLEN and λ , even for a fixed SEW tied to an output type. The type suffix in a long-form multiply-accumulate intrinsic name directly encodes MUL_C , whereas in an overloaded name that MUL_C is implicit. Either way, source code with IME intrinsics is tied to a specific combination of input/output types and value of MUL_C .

Although it is possible to write more general assembly code, it is common industry practice to favor coding with compiler intrinsics. The recommended approach for writing portable code with IME intrinsics is to package multiple code paths in the same executable, each optimized for a specific value of MUL_C . Runtime selection of the appropriate code path is then performed based on the result of `vsetvl` and computations of $MUL_C = VLEN / (SEW \times \lambda^2)$.

36.11. Instructions (in alphabetical order)

The semantics of each instruction is expressed in a SAIL-like syntax, using the following helper functions that are shared across all instructions in this chapter:

```
// Decode a 3-bit lambda encoding to the actual lambda value.
// Returns None() if the encoding is 0b000 (meaning "use vtype.lambda").
function decode_vlambda(lam_enc : bits(3)) -> option(int) =
  if lam_enc == 0b000 then None()
  else Some(2 ^ (unsigned(lam_enc) - 1))

// Map sequential tile element index i to the flat register-group element
// index.
// linesize = lambda * LMUL; elements within one tile row span LMUL registers.
function tile_reg_idx(i : int, LMUL : int, lambda : int, elems_per_reg : int)
-> int = {
  let linesize      : int = lambda * LMUL;
  let line          : int = i / linesize;
  let elem_in_line  : int = i % linesize;
  let regoff        : int = elem_in_line / lambda;
  let elementoff    : int = line * lambda + elem_in_line % lambda;
  regoff * elems_per_reg + elementoff
}

// A[i, k] in vs1: A is stored row-major (rows are the outer dimension).
function mat_A_idx(i : int, k : int, K_eff : int,
                  LMUL : int, lambda : int, epr : int) -> int =
  tile_reg_idx(i * K_eff + k, LMUL, lambda, epr)
```

```

// B[k, j] in vs2: B is stored column-major (columns are the outer dimension).
function mat_B_idx(k : int, j : int, K_eff : int,
                  LMUL : int, lambda : int, epr : int) -> int =
    tile_reg_idx(j * K_eff + k, LMUL, lambda, epr)

// C[i, j] in vd: C is stored column-major.
// MUL_C = VLEN / (SEW * lambda^2) is the number of registers in the C group.
function mat_C_idx(i : int, j : int, M : int,
                  MUL_C : int, lambda : int, epr : int) -> int =
    tile_reg_idx(j * M + i, MUL_C, lambda, epr)

// fp_format_of(EEW : int, alt : bit) -> fp_fmt
//   Decode (element width in bits, altfmt flag) to the concrete FP format as
//   specified
//   by the altfmt encoding tables in the vtype section (e.g. EEW=8, alt=0 →
//   E4M3 (OFP8)).

// fp_mul_to(a      : bits(EEW_A), fmt_A : fp_fmt,
//           b      : bits(EEW_A), fmt_B : fp_fmt,
//           EEW_C : int,      fmt_C : fp_fmt,
//           rm     : rounding_mode) -> bits(EEW_C)
//   Multiply a (in format fmt_A) by b (in format fmt_B), round the product
//   into format fmt_C, and return the EEW_C-wide bit pattern. Updates fflags.
//   Per IEEE 754, the exact (infinite-precision) product  $a \times b$  is formed
//   regardless of whether fmt_A equals fmt_B, then rounded once to fmt_C
//   using rounding mode rm.

// fp_add(acc  : bits(EEW), prod : bits(EEW),
//         EEW  : int,      fmt  : fp_fmt,
//         rm   : rounding_mode) -> bits(EEW)
//   Add prod to acc, both in format fmt at width EEW, rounded per rm. Updates
//   fflags.

// fp_mul(a : bits(EEW), b : bits(EEW),
//        EEW : int,      fmt : fp_fmt,
//        rm  : rounding_mode) -> bits(EEW)
//   Multiply a by b, both in format fmt at width EEW, rounded per rm. Updates
//   fflags.

// fp_one(EEW : int, fmt : fp_fmt) -> bits(EEW)
//   Return the bit pattern for +1.0 in the given format.

// fp_is_NaN(x : bits(EEW), EEW : int, fmt : fp_fmt) -> bool
//   Return true if x is a NaN in the given format.

// fp_defaultNaN(EEW : int, fmt : fp_fmt) -> bits(EEW)
//   Return the default (canonical) NaN bit pattern for the given format.

// fp_zero(EEW : int, fmt : fp_fmt) -> bits(EEW)
//   Return the positive zero bit pattern for the given format.

```

```

// e8m0_to_fp(scale : bits(8), EEW : int, fmt : fp_fmt) -> bits(EEW)
//   Convert an 8-bit E8M0 exponent-only value (bias 127) to the corresponding
//   power-of-two in the target format. E8M0 0xFF maps to NaN.

// int_to_fp(x : int, EEW : int, fmt : fp_fmt, rm : rounding_mode) -> bits(EEW)
//   Convert a mathematical integer x to the nearest representable value in
//   format fmt at width EEW, rounded per rm. Updates fflags.

// — Shared GEMM helpers —————

// Decode tile geometry from vtype for widening factor W.
// Returns a record with all derived tile dimensions.
struct gemm_geom = {
    EEW_C   : int,    // accumulator element width (= SEW)
    EEW_A   : int,    // input element width (= SEW / W)
    LMUL    : int,    // register-group multiplier
    lambda  : int,    // tile-layout parameter from vtype
    K_eff   : int,    // effective K dimension (= lambda * W * LMUL)
    M       : int,    // rows of A / C tile
    N       : int,    // columns of B / C tile (set via VL)
    MUL_C   : int,    // number of registers in C group
    epr_A   : int,    // elements per register at EEW_A
    epr_C   : int,    // elements per register at EEW_C
}

function decode_gemm_geometry(W : int, vl_divisor_is_lambda : bool) ->
gemm_geom = {
    let SEW_pow  : int = get_sew_pow();
    let EEW_C    : int = 2 ^ SEW_pow;
    let EEW_A    : int = EEW_C / W;
    let LMUL_pow : int = get_lmUL_pow();
    let LMUL     : int = 2 ^ max(0, LMUL_pow);
    let epr_A    : int = vlen / EEW_A;
    let epr_C    : int = vlen / EEW_C;

    let lambda : int =
        match decode_vlambda(vtype[lambda]) {
            Some(lam) => lam,
            None()   => return Illegal_Instruction()
        };

    let K_eff : int = lambda * W * LMUL;

    // VL divisibility: integer widening divides by lambda;
    // all other instructions divide by K_eff.
    let vl_div : int = if vl_divisor_is_lambda then lambda else K_eff;
    let N      : int = unsigned(vl) / vl_div;
    let M      : int = (LMUL * vlen / EEW_A) / K_eff;
    let MUL_C  : int = M / lambda;

```

```

if unsigned(vl) % vl_div != 0 then return Illegal_Instruction();
if MUL_C not in {1,2,4,8,16} then return Illegal_Instruction();

struct { EEW_C, EEW_A, LMUL, lambda, K_eff, M, N, MUL_C, epr_A, epr_C }
}

// Read and unpack paired E8M0 block scales from v0 for block index s.
// When vm=1 (no microscaling), returns (1.0, false).
// When vm=0, reads paired 16-bit scale element from v0 and returns
// (scale_A × scale_B, is_nan).
function read_block_scales(vm : bit, i : int, j : int, s : int,
                          R : int, EEW_C : int, fmt_C : fp_fmt,
                          rm : rounding_mode) -> (bits(EEW_C), bool) = {

  let pair_A : bits(16) =
    if vm == 0
    then read_single_element(16, i * R + s, zvreg(0))
    else 0x0000;
  let pair_B : bits(16) =
    if vm == 0
    then read_single_element(16, j * R + s, zvreg(0))
    else 0x0000;

  let sA : bits(EEW_C) =
    if vm == 0
    then e8m0_to_fp(pair_A[7..0], EEW_C, fmt_C)
    else fp_one(EEW_C, fmt_C);
  let sB : bits(EEW_C) =
    if vm == 0
    then e8m0_to_fp(pair_B[15..8], EEW_C, fmt_C)
    else fp_one(EEW_C, fmt_C);

  let blk_scale : bits(EEW_C) = fp_mul(sA, sB, EEW_C, fmt_C, rm);
  (blk_scale, fp_is_NaN(blk_scale, EEW_C, fmt_C))
}

// Floating-point inner product over elements [k_lo, k_hi] with widening.
// Returns the block accumulator in format fmt_C at width EEW_C.
function fp_block_dot(i : int, j : int, k_lo : int, k_hi : int,
                     g : gemm_geom,
                     fmt_A : fp_fmt, fmt_B : fp_fmt, fmt_C : fp_fmt,
                     rm : rounding_mode,
                     vs1 : regidx, vs2 : regidx) -> bits(g.EEW_C) = {
  var blk_acc : bits(g.EEW_C) = fp_zero(g.EEW_C, fmt_C);
  foreach (k from k_lo to k_hi) {
    let a_flat : int = mat_A_idx(i, k, g.K_eff, g.LMUL, g.lambda, g.epr_A);
    let b_flat : int = mat_B_idx(k, j, g.K_eff, g.LMUL, g.lambda, g.epr_A);
    let a_bits : bits(g.EEW_A) = read_single_element(g.EEW_A, a_flat, vs1);
    let b_bits : bits(g.EEW_A) = read_single_element(g.EEW_A, b_flat, vs2);
    let prod : bits(g.EEW_C) = fp_mul_to(a_bits, fmt_A, b_bits, fmt_B,

```



```

                                g.EEW_C, fmt_C, rm);
    blk_acc = fp_add(blk_acc, prod, g.EEW_C, fmt_C, rm)
};
blk_acc
}

// Exact signed integer inner product over elements [k_lo, k_hi].
// Returns an unbounded mathematical integer (no overflow).
function int_block_dot(i : int, j : int, k_lo : int, k_hi : int,
                      g : gemm_geom,
                      signed_A : bool, signed_B : bool,
                      vs1 : regidx, vs2 : regidx) -> int = {
    var int_sum : int = 0;
    foreach (k from k_lo to k_hi) {
        let a_flat : int = mat_A_idx(i, k, g.K_eff, g.LMUL, g.lambda, g.epr_A);
        let b_flat : int = mat_B_idx(k, j, g.K_eff, g.LMUL, g.lambda, g.epr_A);
        let a_val  : int = if signed_A
                        then signed(read_single_element(g.EEW_A, a_flat, vs1))
                        else unsigned(read_single_element(g.EEW_A, a_flat,
vs1));
        let b_val  : int = if signed_B
                        then signed(read_single_element(g.EEW_A, b_flat, vs2))
                        else unsigned(read_single_element(g.EEW_A, b_flat,
vs2));
        int_sum = int_sum + a_val * b_val
    };
    int_sum
}

// Check microscaling legality constraints.
// Raises Illegal_Instruction if  $SEW \times \lambda < 16$  or (for BS=16)  $W \times LMUL > SEW$ .
function check_microscaling_legality(W : int, LMUL : int,
                                    EEW_C : int, lambda : int) -> unit = {
    if EEW_C * lambda < 16 then return Illegal_Instruction();
    if vtype[bs] == 0b1 & (W * LMUL > EEW_C) then return Illegal_Instruction()
}

```

36.11.1. v8wmmacc.vv

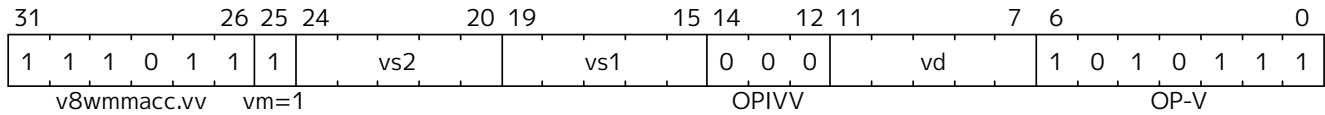
Synopsis

Integer $8 \times$ Widening Matrix Multiply-Accumulate

Mnemonic

v8wmmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Computes the matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$, with a widening factor of 8 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). *vs1* and *vs2* have elements of width $SEW \div 8$; *vd* has elements of width SEW.

$K_{\text{effective}} = \lambda \times 8 \times \text{LMUL}$. VL selects how many columns of B (and C) are updated; VL must be a multiple of λ . The signedness of each input operand is selected by `altfmt_A` and `altfmt_B` in `vtype`: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}).

Only $SEW \in \{32, 64\}$ is valid (giving $EEW = 4$ or 8); $SEW \in \{8, 16\}$ is *reserved*.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- $vm = 0$ (*reserved*).
- $SEW < 32$ ($EEW = SEW \div 8$ would be sub-nibble).
- VL is not a multiple of λ .
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .

Operation

```

if vm == 0 then return Illegal_Instruction();
if (2 ^ get_sew_pow()) < 32 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(8, true);

let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);

```

```
let c_bits : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
var acc : int = signed(c_bits);
acc = acc + int_block_dot(i, j, 0, g.K_eff - 1, g, signed_A, signed_B, vs1,
vs2);
write_single_element(g.EEW_C, c_flat, vd, to_bits_unsafe(g.EEW_C, acc))
}
};
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.11.2. vf8wmmacc.vv

Synopsis

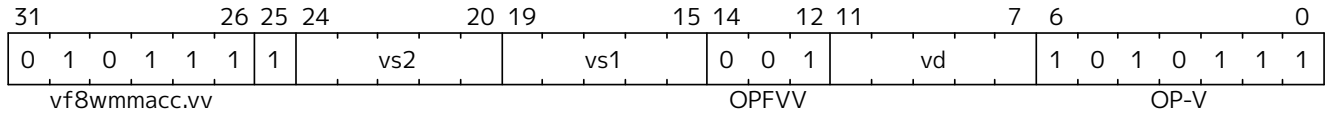
Floating-Point 8× Widening Matrix Multiply-Accumulate

Mnemonic

vf8wmmacc.vv *vd*, *vs1*, *vs2*

vf8wmmacc.vv *vd*, *vs1*, *vs2*, *v0.scale*

Encoding



Description

Computes the floating-point matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$, with a widening factor of 8 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). *vs1* and *vs2* have elements of width $SEW \div 8$; *vd* has elements of width SEW .

$K_{\text{effective}} = \lambda \times 8 \times \text{LMUL}$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_{\text{effective}}$. The floating-point format for A and B elements is selected independently by **altfmt_A** and **altfmt_B**, interpreted for input element width $SEW \div 8$; the C format is selected by **altfmt**, interpreted for width SEW . All operations use the dynamic rounding mode from **frm**; floating-point exception flags accumulate in **fflags**.

When **vm=0** (**v0.scale**), the register **v0** supplies paired E8M0 block-scale factors for microscaling (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

Only $SEW \in \{32, 64\}$ is valid (giving $EEW = 4$ or 8); $SEW \in \{8, 16\}$ is reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- $SEW < 32$ ($EEW = SEW \div 8$ would be sub-nibble).
- VL is not a multiple of $K_{\text{effective}} (= \lambda \times 8 \times \text{LMUL})$.
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .
- When **vm=0**: $SEW \times \lambda < 16$.
- When **vm=0** and **bs=1**: $W \times \text{LMUL} > SEW$ (i.e., $8 \times \text{LMUL} > SEW$).
- When **vm=0**: *vd*, *vs1*, or *vs2* overlaps **v0**.

Operation

```
if (2 ^ get_sew_pow()) < 32 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(8, false);
```

```

// Microscaling checks (only apply when vm=0)
if vm == 0 then check_microscaling_legality(8, g.LMUL, g.EEW_C, g.lambda);

let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
let S : int = (g.K_eff + block_size - 1) / block_size;
let R : int = g.lambda * g.EEW_C / 16;

let fmt_A : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm      : rounding_mode = get_fp_rounding_mode();

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int      = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.
epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var nan_out : bool = false;

    foreach (s from 0 to (S - 1)) {
      let (blk_scale, is_nan) = read_block_scales(vm, i, j, s, R,
g.EEW_C, fmt_C, rm);

      if is_nan then { nan_out = true; break };

      let k_lo : int = s * block_size;
      let k_hi : int = min(k_lo + block_size, g.K_eff) - 1;
      let blk_acc : bits(g.EEW_C) = fp_block_dot(i, j, k_lo, k_hi, g,
fmt_A, fmt_B, fmt_C, rm,
vs1, vs2);

      acc = fp_add(acc, fp_mul(blk_scale, blk_acc, g.EEW_C, fmt_C, rm),
g.EEW_C, fmt_C, rm)
    };

    if nan_out then
      write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C, fmt_C))
    else
      write_single_element(g.EEW_C, c_flat, vd, acc)
  }
};
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvfmm (Matrix-multiplication instructions (floating-point))	0.1	Draft

36.11.3. vfmacc.vv

Synopsis

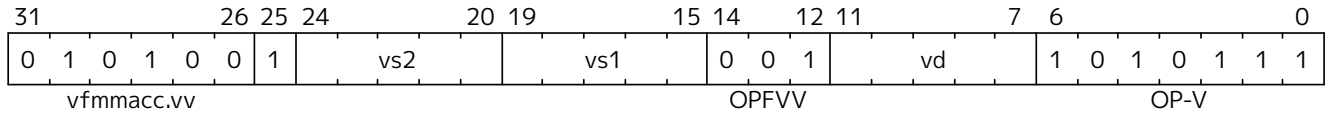
Floating-Point Matrix Multiply-Accumulate

Mnemonic

vfmacc.vv *vd*, *vs1*, *vs2*

vfmacc.vv *vd*, *vs1*, *vs2*, *vs0.scale*

Encoding



Description

Computes the floating-point matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). All three tiles have elements of width SEW.

$K_{\text{effective}} = \lambda \times \text{LMUL}$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_{\text{effective}}$. The floating-point format for A and B elements is selected independently by `altfmt_A` and `altfmt_B` in `vtype` (see [Section 36.7.1](#)); the C accumulator format is selected by `altfmt`. All operations use the dynamic rounding mode from `frm`; floating-point exception flags accumulate in `fflags`.

Microscaling (`vm=0`) is not supported for non-widening floating-point multiply-accumulate; `vm=0` is reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- `vm = 0` (reserved).
- VL is not a multiple of $K_{\text{effective}}$ ($= \lambda \times \text{LMUL}$).
- $\text{MUL_C} (= \text{VLEN} \div (\text{SEW} \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .

Operation

```
if vm == 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(1, false);

let fmt_A : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm      : rounding_mode = get_fp_rounding_mode();

foreach (j from 0 to (g.N - 1)) {
```

```
foreach (i from 0 to (g.M - 1)) {
  let c_flat : int      = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);
  var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
  acc = fp_add(acc, fp_block_dot(i, j, 0, g.K_eff - 1, g,
                                fmt_A, fmt_B, fmt_C, rm, vs1, vs2),
              g.EEW_C, fmt_C, rm);
  write_single_element(g.EEW_C, c_flat, vd, acc)
}
};
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvvfmm (Matrix-multiplication instructions (floating-point))	0.1	Draft
Zvvxofp8mm, Zvvxnofp8mm (Section 36.7.3)	0.1	Draft

36.11.4. vfwmmacc.vv

Synopsis

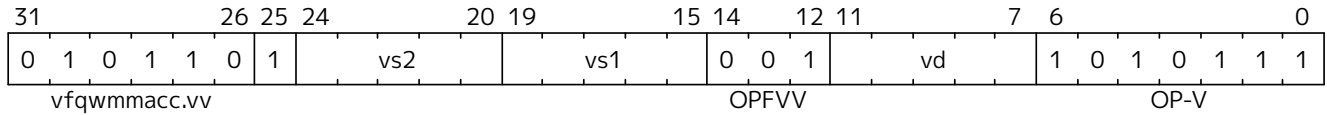
Floating-Point Quad-Widening Matrix Multiply-Accumulate

Mnemonic

vfwmmacc.vv *vd*, *vs1*, *vs2*

vfwmmacc.vv *vd*, *vs1*, *vs2*, *v0.scale*

Encoding



Description

Computes the floating-point matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$, with a widening factor of 4 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). *vs1* and *vs2* have elements of width $SEW \div 4$; *vd* has elements of width SEW .

$K_{\text{effective}} = \lambda \times 4 \times LMUL$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_{\text{effective}}$. The floating-point format for A and B elements is selected independently by **altfmt_A** and **altfmt_B**, interpreted for input element width $SEW \div 4$; the C format is selected by **altfmt**, interpreted for width SEW . All operations use the dynamic rounding mode from **frm**; floating-point exception flags accumulate in **fflags**.

When **vm=0** (**v0.scale**), the register **v0** supplies paired E8M0 block-scale factors for microscaling (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- VL is not a multiple of $K_{\text{effective}}$ ($= \lambda \times 4 \times LMUL$).
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .
- When **vm=0**: $SEW \times \lambda < 16$.
- When **vm=0** and **bs=1**: $W \times LMUL > SEW$.
- When **vm=0**: *vd*, *vs1*, or *vs2* overlaps **v0**.

Operation

```
let g : gemm_geom = decode_gemm_geometry(4, false);

// Microscaling checks (only apply when vm=0)
if vm == 0 then check_microscaling_legality(4, g.LMUL, g.EEW_C, g.lambda);

let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
```



```

let S : int = (g.K_eff + block_size - 1) / block_size;
let R : int = g.lambda * g.EEW_C / 16;

let fmt_A : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm    : rounding_mode = get_fp_rounding_mode();

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int      = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.
epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var nan_out : bool = false;

    foreach (s from 0 to (S - 1)) {
      let (blk_scale, is_nan) = read_block_scales(vm, i, j, s, R,
                                                g.EEW_C, fmt_C, rm);

      if is_nan then { nan_out = true; break };

      let k_lo : int = s * block_size;
      let k_hi : int = min(k_lo + block_size, g.K_eff) - 1;
      let blk_acc : bits(g.EEW_C) = fp_block_dot(i, j, k_lo, k_hi, g,
                                                fmt_A, fmt_B, fmt_C, rm,
                                                vs1, vs2);

      acc = fp_add(acc, fp_mul(blk_scale, blk_acc, g.EEW_C, fmt_C, rm),
                  g.EEW_C, fmt_C, rm)
    };

    if nan_out then
      write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C, fmt_C))
    else
      write_single_element(g.EEW_C, c_flat, vd, acc)
  }
};
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvpmm (Matrix-multiplication instructions (floating-point))	0.1	Draft
Zvvxofp4fp16mm, Zvvxofp4bfp16mm, Zvvxofp8fp32mm (Section 36.7.3)	0.1	Draft
Zvvxnofp4fp16mm, Zvvxnofp4bfp16mm, Zvvxnofp8fp32mm (Section 36.7.3)	0.1	Draft

36.11.5. vfwmmacc.vv

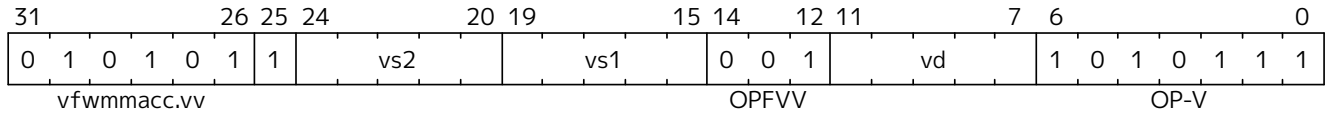
Synopsis

Floating-Point Widening Matrix Multiply-Accumulate

Mnemonic

vfwmmacc.vv *vd*, *vs1*, *vs2*
vfwmmacc.vv *vd*, *vs1*, *vs2*, *v0.scale*

Encoding



Description

Computes the floating-point matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$, with a widening factor of 2 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). *vs1* and *vs2* have elements of width $SEW \div 2$; *vd* has elements of width SEW .

$K_{\text{effective}} = \lambda \times 2 \times LMUL$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_{\text{effective}}$. The floating-point format for A and B elements is selected independently by **altfmt_A** and **altfmt_B**, interpreted for input element width $SEW \div 2$; the C format is selected by **altfmt**, interpreted for width SEW . All operations use the dynamic rounding mode from **frm**; floating-point exception flags accumulate in **fflags**.

When **vm=0** (**v0.scale**), the register **v0** supplies paired E8M0 block-scale factors for microscaling (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- VL is not a multiple of $K_{\text{effective}}$ ($= \lambda \times 2 \times LMUL$).
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .
- When **vm=0**: $SEW \times \lambda < 16$.
- When **vm=0** and **bs=1**: $W \times LMUL > SEW$.
- When **vm=0**: *vd*, *vs1*, or *vs2* overlaps **v0**.

Operation

```
let g : gemm_geom = decode_gemm_geometry(2, false);

// Microscaling checks (only apply when vm=0)
if vm == 0 then check_microscaling_legality(2, g.LMUL, g.EEW_C, g.lambda);

let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
```

```

let S : int = (g.K_eff + block_size - 1) / block_size;
let R : int = g.lambda * g.EEW_C / 16;

let fmt_A : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_A]);
let fmt_B : fp_fmt      = fp_format_of(g.EEW_A, vtype[altfmt_B]);
let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm    : rounding_mode = get_fp_rounding_mode();

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int      = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.
epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var nan_out : bool = false;

    foreach (s from 0 to (S - 1)) {
      let (blk_scale, is_nan) = read_block_scales(vm, i, j, s, R,
                                                g.EEW_C, fmt_C, rm);

      if is_nan then { nan_out = true; break };

      let k_lo : int = s * block_size;
      let k_hi : int = min(k_lo + block_size, g.K_eff) - 1;
      let blk_acc : bits(g.EEW_C) = fp_block_dot(i, j, k_lo, k_hi, g,
                                                fmt_A, fmt_B, fmt_C, rm,
                                                vs1, vs2);

      acc = fp_add(acc, fp_mul(blk_scale, blk_acc, g.EEW_C, fmt_C, rm),
                  g.EEW_C, fmt_C, rm)
    };

    if nan_out then
      write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C, fmt_C))
    else
      write_single_element(g.EEW_C, c_flat, vd, acc)
    }
  };
}
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvpmm (Matrix-multiplication instructions (floating-point))	0.1	Draft
Zvvxofp4ofp8mm, Zvvxofp8fp16mm, Zvvxofp8bfp16mm (Section 36.7.3)	0.1	Draft
Zvvxnofp4ofp8mm, Zvvxnofp8fp16mm, Zvvxnofp8bfp16mm (Section 36.7.3)	0.1	Draft

36.11.6. vfwimmac.vv

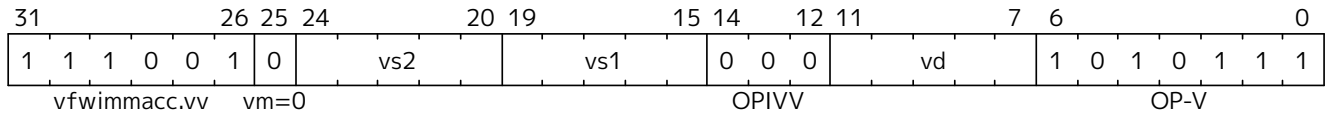
Synopsis

Integer-Input Widening Floating-Point-Accumulate Matrix Multiply-Accumulate with E8MO Microscaling

Mnemonic

vfwimmac.vv vd, vs1, vs2, v0.scale

Encoding



This instruction shares its opcode with **vmmacc.vv**; **vm=0** selects the integer-input FP-accumulate MX form. **vmmacc.vv** requires **vm=1**.

Description

Computes the exact integer matrix-matrix product $T = vs1 \times vs2$ block by block, scales each block by the paired E8MO factors in **v0**, and accumulates the scaled result into the floating-point accumulator **vd**: $C \leftarrow C + scale_A \times scale_B \times T$. The widening factor is 2 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), **vs2** as a $K \times N$ tile (column-major register layout), and **vd** as an $M \times N$ FP accumulator tile (column-major register layout). **vs1** and **vs2** have integer elements of width $SEW \div 2$; **vd** has FP elements of width SEW .

$K_effective = \lambda \times 2 \times LMUL$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_effective$. MXINT inputs are always signed; **altfmt_A** and **altfmt_B** must be 0. **altfmt** selects the FP format of the accumulator C.

The register **v0** supplies paired E8MO block-scale factors using the same tile-strided layout as **vfwimmac.vv** (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

The encodings $SEW=8$ ($EEW=4$, Int4 inputs into OFP8 accumulator), $SEW=32$, and $SEW=64$ are reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- $SEW = 8$ (reserved encoding).
- $SEW = 32$ (reserved encoding).
- $SEW = 64$ (reserved encoding).
- **altfmt_A** = 1 (reserved; MXINT inputs are signed).
- **altfmt_B** = 1 (reserved; MXINT inputs are signed).
- VL is not a multiple of $K_effective (= \lambda \times 2 \times LMUL)$.
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- **vs1** or **vs2** is not aligned for LMUL.
- **vd** is not aligned for MUL_C .
- $SEW \times \lambda < 16$.

- When $bs=1$: $W \times LMUL > SEW$ (i.e., $2 \times LMUL > SEW$).
- vd , $vs1$, or $vs2$ overlaps $v0$.

Operation

```

let EEW_C_check : int = 2 ^ get_sew_pow();
// SEW=8 (Int4 → OFP8) is reserved
if EEW_C_check == 8 then return Illegal_Instruction();
// SEW=32 is reserved
if EEW_C_check == 32 then return Illegal_Instruction();
// SEW=64 is reserved
if EEW_C_check == 64 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(2, false);

check_microscaling_legality(2, g.LMUL, g.EEW_C, g.lambda);

// MXINT inputs are always signed
if vtype[altfmt_A] == 0b1 then return Illegal_Instruction();
if vtype[altfmt_B] == 0b1 then return Illegal_Instruction();

let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
let S : int = (g.K_eff + block_size - 1) / block_size;
let R : int = g.lambda * g.EEW_C / 16;

let fmt_C : fp_fmt = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm : rounding_mode = get_fp_rounding_mode();

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var nan_out : bool = false;

    foreach (s from 0 to (S - 1)) {
      let (blk_scale, is_nan) = read_block_scales(0b0, i, j, s, R,
                                                g.EEW_C, fmt_C, rm);

      if is_nan then { nan_out = true; break };

      let k_lo : int = s * block_size;
      let k_hi : int = min(k_lo + block_size, g.K_eff) - 1;
      let dot : int = int_block_dot(i, j, k_lo, k_hi, g, true, true, vs1,
vs2);
      let fp_sum : bits(g.EEW_C) = int_to_fp(dot, g.EEW_C, fmt_C, rm);
      acc = fp_add(acc, fp_mul(blk_scale, fp_sum, g.EEW_C, fmt_C, rm),
                  g.EEW_C, fmt_C, rm)
    };

    if nan_out then
      write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C, fmt_C))
  }
}

```

```
    else
        write_single_element(g.EEW_C, c_flat, vd, acc)
    }
};
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvvxi8fp16mm, Zvvxi8bf16mm (Section 36.7.3)	0.1	Draft
Zvvxni8fp16mm, Zvvxni8bf16mm (Section 36.7.3)	0.1	Draft

36.11.7. vf8wimmacc.vv

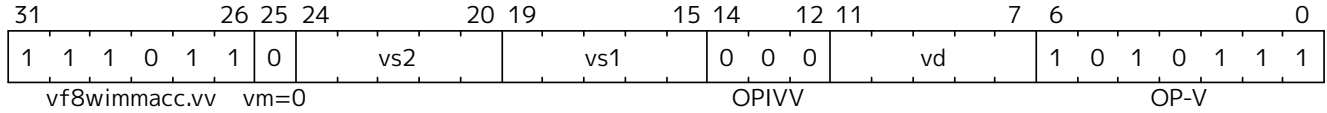
Synopsis

Integer-Input 8× Widening Floating-Point-Accumulate Matrix Multiply-Accumulate with E8MO Microscaling

Mnemonic

vf8wimmacc.vv vd, vs1, vs2, v0.scale

Encoding



This instruction shares its opcode with **v8wmmacc.vv**; **vm=0** selects the integer-input FP-accumulate MX form. **v8wmmacc.vv** requires **vm=1**.

Description

Computes the exact integer matrix-matrix product $T = vs1 \times vs2$ block by block, scales each block by the paired E8MO factors in **v0**, and accumulates the scaled result into the floating-point accumulator **vd**: $C \leftarrow C + scale_A \times scale_B \times T$. The widening factor is 8 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), **vs2** as a $K \times N$ tile (column-major register layout), and **vd** as an $M \times N$ FP accumulator tile (column-major register layout). **vs1** and **vs2** have integer elements of width $SEW \div 8$; **vd** has FP elements of width SEW .

$K_effective = \lambda \times 8 \times LMUL$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_effective$. MXINT inputs are always signed; **altfmt_A** and **altfmt_B** must be 0. **altfmt** selects the FP format of the accumulator C.

The register **v0** supplies paired E8MO block-scale factors using the same tile-strided layout as **vf8wmmacc.vv** (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

Only $SEW \in \{32, 64\}$ is valid; $SEW \in \{8, 16\}$ is reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- $SEW < 32$ (reserved encoding).
- **altfmt_A** = 1 (reserved; MXINT inputs are signed).
- **altfmt_B** = 1 (reserved; MXINT inputs are signed).
- VL is not a multiple of $K_effective (= \lambda \times 8 \times LMUL)$.
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- **vs1** or **vs2** is not aligned for LMUL.
- **vd** is not aligned for MUL_C .
- $SEW \times \lambda < 16$.
- When **bs=1**: $W \times LMUL > SEW$ (i.e., $8 \times LMUL > SEW$).
- **vd**, **vs1**, or **vs2** overlaps **v0**.

Operation

```

if (2 ^ get_sew_pow()) < 32 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(8, false);

check_microscaling_legality(8, g.LMUL, g.EEW_C, g.lambda);

// MXINT inputs are always signed
if vtype[altfmt_A] == 0b1 then return Illegal_Instruction();
if vtype[altfmt_B] == 0b1 then return Illegal_Instruction();

let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
let S : int = (g.K_eff + block_size - 1) / block_size;
let R : int = g.lambda * g.EEW_C / 16;

let fmt_C : fp_fmt      = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm      : rounding_mode = get_fp_rounding_mode();

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int      = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var nan_out : bool = false;

    foreach (s from 0 to (S - 1)) {
      let (blk_scale, is_nan) = read_block_scales(0b0, i, j, s, R,
                                                g.EEW_C, fmt_C, rm);

      if is_nan then { nan_out = true; break };

      let k_lo : int = s * block_size;
      let k_hi : int = min(k_lo + block_size, g.K_eff) - 1;
      let dot  : int = int_block_dot(i, j, k_lo, k_hi, g, true, true, vs1,
vs2);
      let fp_sum : bits(g.EEW_C) = int_to_fp(dot, g.EEW_C, fmt_C, rm);
      acc = fp_add(acc, fp_mul(blk_scale, fp_sum, g.EEW_C, fmt_C, rm),
                  g.EEW_C, fmt_C, rm)
    };

    if nan_out then
      write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C, fmt_C))
    else
      write_single_element(g.EEW_C, c_flat, vd, acc)
    }
  };
};
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvxi4fp32mm, Zvvxi8fp64mm (Section 36.7.3)	0.1	Draft
Zvvxni4fp32mm, Zvvxni8fp64mm (Section 36.7.3)	0.1	Draft

36.11.8. vfqwimmac.vv

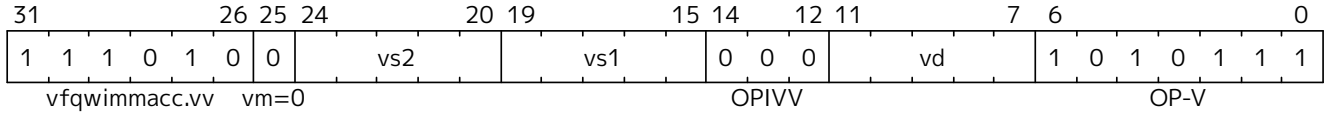
Synopsis

Integer-Input Quad-Widening Floating-Point-Accumulate Matrix Multiply-Accumulate with E8MO Microscaling

Mnemonic

vfqwimmac.vv vd, vs1, vs2, v0.scale

Encoding



This instruction shares its opcode with **vqwmacc.vv**; **vm=0** selects the integer-input FP-accumulate MX form. **vqwmacc.vv** requires **vm=1**.

Description

Computes the exact integer matrix-matrix product $T = vs1 \times vs2$ block by block, scales each block by the paired E8MO factors in **v0**, and accumulates the scaled result into the floating-point accumulator **vd**: $C \leftarrow C + scale_A \times scale_B \times T$. The widening factor is 4 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), **vs2** as a $K \times N$ tile (column-major register layout), and **vd** as an $M \times N$ FP accumulator tile (column-major register layout). **vs1** and **vs2** have integer elements of width $SEW \div 4$; **vd** has FP elements of width SEW .

$K_effective = \lambda \times 4 \times LMUL$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_effective$. MXINT inputs are always signed; **altfmt_A** and **altfmt_B** must be 0. **altfmt** selects the FP format of the accumulator C.

The register **v0** supplies paired E8MO block-scale factors using the same tile-strided layout as **vfqwmacc.vv** (see [Section 36.7.3](#)). The **bs** field in **vtype** selects the block size (32 or 16 elements).

The encodings $SEW=8$ ($EEW=2$), $altfmt=1$ with $SEW=32$, and $SEW=64$ are reserved.

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- $SEW = 8$ (reserved encoding).
- $SEW = 32$ and **altfmt** = 1 (reserved).
- $SEW = 64$ (reserved encoding).
- **altfmt_A** = 1 (reserved; MXINT inputs are signed).
- **altfmt_B** = 1 (reserved; MXINT inputs are signed).
- VL is not a multiple of $K_effective$ ($= \lambda \times 4 \times LMUL$).
- MUL_C ($= VLEN \div (SEW \times \lambda^2)$) is not in $\{1, 2, 4, 8, 16\}$.
- **vs1** or **vs2** is not aligned for LMUL.
- **vd** is not aligned for MUL_C .
- $SEW \times \lambda < 16$.
- When **bs=1**: $W \times LMUL > SEW$ (i.e., $4 \times LMUL > SEW$).

- *vd*, *vs1*, or *vs2* overlaps *v0*.

Operation

```

let EEW_C_check : int = 2 ^ get_sew_pow();
// SEW=8 (EEW=2) is reserved
if EEW_C_check == 8 then return Illegal_Instruction();
// SEW=32 with altfmt=1 is reserved
if EEW_C_check == 32 & vtype[altfmt] == 0b1 then return Illegal_Instruction();
// SEW=64 is reserved
if EEW_C_check == 64 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(4, false);

check_microscaling_legality(4, g.LMUL, g.EEW_C, g.lambda);

// MXINT inputs are always signed
if vtype[altfmt_A] == 0b1 then return Illegal_Instruction();
if vtype[altfmt_B] == 0b1 then return Illegal_Instruction();

let block_size : int = if vtype[bs] == 0b1 then 16 else 32;
let S : int = (g.K_eff + block_size - 1) / block_size;
let R : int = g.lambda * g.EEW_C / 16;

let fmt_C : fp_fmt = fp_format_of(g.EEW_C, vtype[altfmt]);
let rm : rounding_mode = get_fp_rounding_mode();

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);
    var acc : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var nan_out : bool = false;

    foreach (s from 0 to (S - 1)) {
      let (blk_scale, is_nan) = read_block_scales(0b0, i, j, s, R,
                                                g.EEW_C, fmt_C, rm);

      if is_nan then { nan_out = true; break };

      let k_lo : int = s * block_size;
      let k_hi : int = min(k_lo + block_size, g.K_eff) - 1;
      let dot : int = int_block_dot(i, j, k_lo, k_hi, g, true, true, vs1,
vs2);
      let fp_sum : bits(g.EEW_C) = int_to_fp(dot, g.EEW_C, fmt_C, rm);
      acc = fp_add(acc, fp_mul(blk_scale, fp_sum, g.EEW_C, fmt_C, rm),
                  g.EEW_C, fmt_C, rm)
    };

    if nan_out then
      write_single_element(g.EEW_C, c_flat, vd, fp_defaultNaN(g.EEW_C, fmt_C))
    else

```

```
        write_single_element(g.EEW_C, c_flat, vd, acc)
    }
};
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvvxi4fp16mm, Zvvxi4bf16mm, Zvvxi8fp32mm (Section 36.7.3)	0.1	Draft
Zvvxni4fp16mm, Zvvxni4bf16mm, Zvvxni8fp32mm (Section 36.7.3)	0.1	Draft

36.11.9. vmmacc.vv

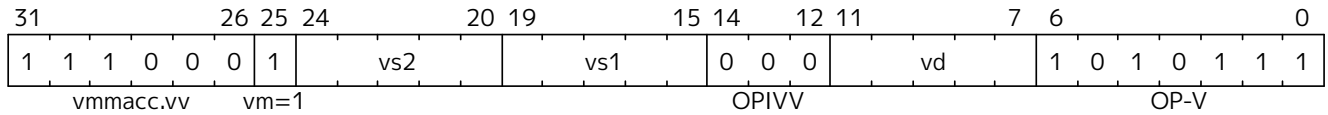
Synopsis

Integer Matrix Multiply-Accumulate

Mnemonic

vmmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Computes the matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). All three tiles have elements of width SEW.

$K_effective = \lambda \times LMUL$. VL selects how many columns of B (and C) are updated; VL must be a multiple of $K_effective$. The signedness of each input operand is selected by `altfmt_A` and `altfmt_B` in `vtype`: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}).

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- `vm = 0` (*reserved*).
- VL is not a multiple of $K_effective (= \lambda \times LMUL)$.
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .

Operation

```

if vm == 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(1, false);

let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);
    let c_bits : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var acc : int = signed(c_bits);
    acc = acc + int_block_dot(i, j, 0, g.K_eff - 1, g, signed_A, signed_B, vs1,
vs2);
    write_single_element(g.EEW_C, c_flat, vd, to_bits_unsafe(g.EEW_C, acc))
  }
}

```

```
}  
};  
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.11.10. vmtl.v

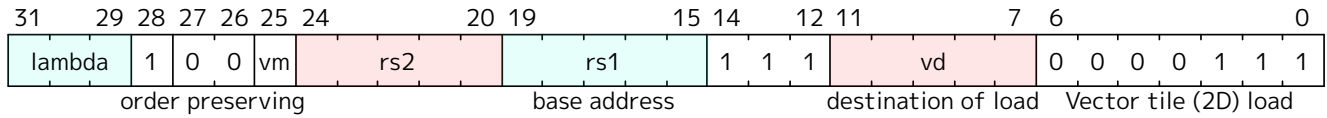
Synopsis

Order-Preserving Matrix Tile Load

Mnemonic

vmtl.v *vd*, (*rs1*), *rs2* [, *Lλ*] [, *vm*]

Encoding



Description

Loads a 2D matrix tile from memory into the vector register group starting at *vd*. *rs1* holds the matrix base address; *rs2* holds the leading dimension **LD** in elements. if *rs2* holds zero (0), the leading dimension **LD** is set to $\lambda \times \text{LMUL}$. When *Lλ* is specified, it overrides `lambda[2:0]` from `vtype` for this instruction. Inactive elements (masked off) are not updated unless `vtype.vma=1` (mask-agnostic), in which case they may be overwritten with 1s. Tail elements (index $\geq \text{VL}$) are not updated unless `vtype.vta=1` (tail-agnostic), in which case they may be overwritten with 1s. Neither inactive nor tail elements raise memory exceptions.

Let $\text{linesize} = \lambda \times \text{LMUL}$. For each element index *i* in the body [*vstart*, *VL*) where the mask is enabled:

$$\text{VD}[i] = \text{M}[\text{rs1} + (i / \text{linesize}) \times \text{LD} + (i \% \text{linesize})]$$

This is the correct load variant when A is stored in row-major order or when B is stored in column-major order: in both cases the memory layout consists of *linesize*-element contiguous segments with a stride of **LD** elements between them. For **LMUL**=1 this simplifies to λ -element segments.

Operation

```

let SEW_pow      : int      = get_sew_pow();
let EEW          : int      = 2 ^ SEW_pow;
let EEW_bytes    : word_width = 2 ^ (SEW_pow - 3);
let LMUL_pow     : int      = get_lmUL_pow();
let LMUL         : int      = 2 ^ max(0, LMUL_pow);
let elems_per_reg : int      = vlen / EEW;
let num_elem     : int      = unsigned(vl);

let eff_lambda : int =
  match decode_vlambda(instr.lambda) {
    Some(lam) => lam,
    None()    => match decode_vlambda(vtype[lambda]) {
      Some(lam) => lam,
      None()    => return Illegal_Instruction()
    }
  }
};

```

```

let LD      : int      = unsigned(X(rs2));
let vm_val  = read_vmask(num_elem, vm, zvreg);
let linesize : int      = eff_lambda * LMUL;

if LD == 0 then {
  LD = eff_lambda * LMUL;
}

foreach (i from 0 to (num_elem - 1)) {
  if vm_val[i] == 0b1 then {
    set_vstart(to_bits_unsafe(16, i));
    let flat_idx : int = tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);
    let mem_off  : int = (i / linesize) * LD + (i % linesize);
    match vmem_read(rs1, to_bits_unsafe(xlen, mem_off * EEW_bytes),
                    EEW_bytes, Load(Data), false, false, false) {
      Ok(elem) => write_single_element(EEW, flat_idx, vd, elem),
      Err(e)   => return e,
    }
  }
}
// inactive (body, mask=0): undisturbed unless vma=1 (may overwrite with 1s)
// tail (i >= VL): not reached by this loop; undisturbed unless vta=1 (may
overwrite with 1s)
};

set_vstart(zeros());
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvvmtnls (Load/store instructions for matrix tiles)	0.1	Draft

36.11.11. vmts.v

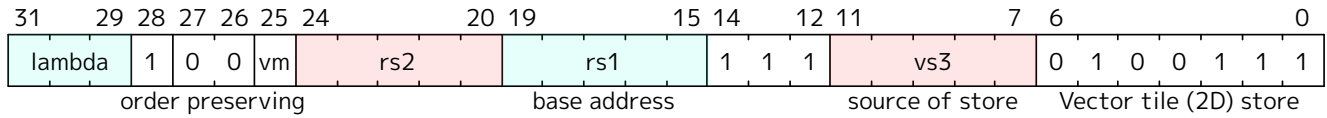
Synopsis

Order-Preserving Matrix Tile Store

Mnemonic

vmts.v vs3, (rs1), rs2 [, Lλ] [, vm]

Encoding



Description

Stores the 2D matrix tile held in the vector register group starting at vs3 to memory. rs1 holds the matrix base address; rs2 holds the leading dimension LD in elements. if rs2 holds zero (0), the leading dimension LD is set to $\lambda \times \text{LMUL}$. When $L\lambda$ is specified, it overrides lambda[2:0] from vtype for this instruction. Inactive elements (masked off) are not written to memory and do not raise exceptions. Tail elements (index $\geq \text{VL}$) are not written to memory and do not raise exceptions.

Let $\text{linesize} = \lambda \times \text{LMUL}$. For each element index i in the body [vstart, VL) where the mask is enabled:

$$M[\text{rs1} + (i / \text{linesize}) \times \text{LD} + (i \% \text{linesize})] = \text{VS}[i]$$

Operation

```

let SEW_pow      : int      = get_sew_pow();
let EEW          : int      = 2 ^ SEW_pow;
let EEW_bytes    : word_width = 2 ^ (SEW_pow - 3);
let LMUL_pow     : int      = get_lmUL_pow();
let LMUL         : int      = 2 ^ max(0, LMUL_pow);
let elems_per_reg : int      = vlen / EEW;
let num_elem     : int      = unsigned(vl);

let eff_lambda : int =
  match decode_vlambda(instr.lambda) {
    Some(lam) => lam,
    None()    => match decode_vlambda(vtype[lambda]) {
      Some(lam) => lam,
      None()    => return Illegal_Instruction()
    }
  };

let LD      : int = unsigned(X(rs2));
let vm_val  : int = read_vmask(num_elem, vm, zvreg);
let linesize : int = eff_lambda * LMUL;

if LD == 0 then {
  LD = eff_lambda * LMUL;

```

```

}

foreach (i from 0 to (num_elem - 1)) {
  if vm_val[i] == 0b1 then {
    set_vstart(to_bits_unsafe(16, i));
    let flat_idx : int = tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);
    let mem_off  : int = (i / linesize) * LD + (i % linesize);
    let data      = read_single_element(EEW, flat_idx, vs3);
    match vmem_write(rs1, to_bits_unsafe(xlen, mem_off * EEW_bytes),
                     EEW_bytes, data, Store(Data), false, false, false) {
      Ok(true)  => (),
      Ok(false) => internal_error(__FILE__, __LINE__, "store got false from
vmem_write"),
      Err(e)    => return e,
    }
  }
};

set_vstart(zeros());
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvmmtls (Load/store instructions for matrix tiles)	0.1	Draft

36.11.12. vmttl.v

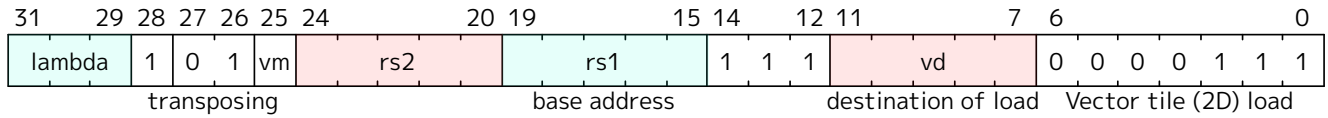
Synopsis

Transposing Matrix Tile Load

Mnemonic

vmttl.v *vd*, (*rs1*), *rs2* [, *Lλ*] [, *vm*]

Encoding



Description

Loads a 2D matrix tile from memory and transposes it into the vector register layout required by the matrix multiplier. This instruction is used when a B tile is stored in row-major order, or when an A or C tile is stored in column-major order in memory. *rs1* holds the matrix base address; *rs2* holds the leading dimension **LD** in elements. if *rs2* holds zero (0), the leading dimension **LD** is set to $\lambda \times \text{LMUL}$. When *Lλ* is specified, it overrides **lambda[2:0]** from **vtype** for this instruction. Inactive elements (masked off) are not updated unless **vtype.vma**=1 (mask-agnostic), in which case they may be overwritten with 1s. Tail elements (index $\geq \text{VL}$) are not updated unless **vtype.vta**=1 (tail-agnostic), in which case they may be overwritten with 1s. Neither inactive nor tail elements raise memory exceptions.

Let $\text{linesize} = \lambda \times \text{LMUL}$. For each element index *i* in the body [**vstart**, **VL**) where the mask is enabled:

```
VD[i] = M[rs1 + (i % linesize) × LD + (i / linesize)]
```

Operation

```
let SEW_pow      : int      = get_sew_pow();
let EEW          : int      = 2 ^ SEW_pow;
let EEW_bytes    : word_width = 2 ^ (SEW_pow - 3);
let LMUL_pow     : int      = get_lmUL_pow();
let LMUL         : int      = 2 ^ max(0, LMUL_pow);
let elems_per_reg : int      = vlen / EEW;
let num_elem     : int      = unsigned(vl);

let eff_lambda : int =
  match decode_vlambda(instr.lambda) {
    Some(lam) => lam,
    None()    => match decode_vlambda(vtype[lambda]) {
      Some(lam) => lam,
      None()    => return Illegal_Instruction()
    }
  };

let LD      : int = unsigned(X(rs2));
let vm_val  = read_vmask(num_elem, vm, zvreg);
let linesize : int = eff_lambda * LMUL;
```

```

if LD == 0 then {
    LD = eff_lambda * LMUL;
}

foreach (i from 0 to (num_elem - 1)) {
    if vm_val[i] == 0b1 then {
        set_vstart(to_bits_unsafe(16, i));
        let flat_idx : int = tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);
        // Transposing: swap line and elem_in_line for the memory address
        let mem_off  : int = (i % linesize) * LD + (i / linesize);
        match vmem_read(rs1, to_bits_unsafe(xlen, mem_off * EEW_bytes),
                      EEW_bytes, Load(Data), false, false, false) {
            Ok(elem) => write_single_element(EEW, flat_idx, vd, elem),
            Err(e)   => return e,
        }
    }
}
// inactive (body, mask=0): undisturbed unless vma=1 (may overwrite with 1s)
// tail (i >= VL): not reached by this loop; undisturbed unless vta=1 (may
// overwrite with 1s)
};

set_vstart(zeros());
RETIRE_SUCCESS

```

Included in

Extension	Minimum version	Lifecycle state
Zvmmmtls (Transposing load/store instructions for matrix tiles)	0.1	Draft

36.11.13. vmtts.v

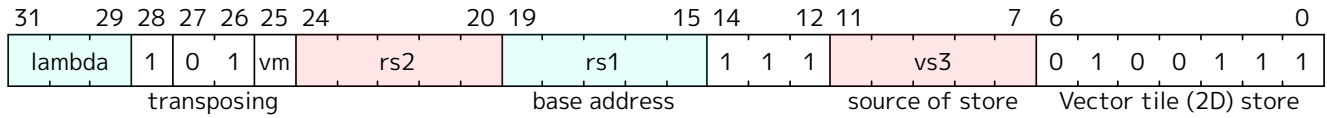
Synopsis

Transposing Matrix Tile Store

Mnemonic

vmtts.v vs3, (rs1), rs2 [, Lλ] [, vm]

Encoding



Description

Stores a 2D matrix tile from vector registers to memory, applying the inverse transposition of `vmttl.v`. `rs1` holds the matrix base address; `rs2` holds the leading dimension **LD** in elements. if `rs2` holds zero (0), the leading dimension **LD** is set to $\lambda \times \text{LMUL}$. When $L\lambda$ is specified, it overrides `lambda[2:0]` from `vtype` for this instruction. Inactive elements (masked off) are not written to memory and do not raise exceptions. Tail elements (index $\geq \text{VL}$) are not written to memory and do not raise exceptions.

Let $\text{linesize} = \lambda \times \text{LMUL}$. For each element index i in the body [`vstart`, `VL`) where the mask is enabled:

$$M[\text{rs1} + (i \% \text{linesize}) \times \text{LD} + (i / \text{linesize})] = \text{VS}[i]$$

Operation

```

let SEW_pow      : int      = get_sew_pow();
let EEW          : int      = 2 ^ SEW_pow;
let EEW_bytes    : word_width = 2 ^ (SEW_pow - 3);
let LMUL_pow     : int      = get_lmUL_pow();
let LMUL         : int      = 2 ^ max(0, LMUL_pow);
let elems_per_reg : int      = vlen / EEW;
let num_elem     : int      = unsigned(vl);

let eff_lambda : int =
  match decode_vlambda(instr.lambda) {
    Some(lam) => lam,
    None()    => match decode_vlambda(vtype[lambda]) {
      Some(lam) => lam,
      None()    => return Illegal_Instruction()
    }
  };

let LD      : int = unsigned(X(rs2));
let vm_val  : int = read_vmask(num_elem, vm, zvreg);
let linesize : int = eff_lambda * LMUL;

if LD == 0 then {
  LD = eff_lambda * LMUL;

```

```
}

foreach (i from 0 to (num_elem - 1)) {
  if vm_val[i] == 0b1 then {
    set_vstart(to_bits_unsafe(16, i));
    let flat_idx : int = tile_reg_idx(i, LMUL, eff_lambda, elems_per_reg);
    // Transposing: swap line and elem_in_line for the memory address
    let mem_off  : int = (i % linesize) * LD + (i / linesize);
    let data
      = read_single_element(EEW, flat_idx, vs3);
    match vmem_write(rs1, to_bits_unsafe(xlen, mem_off * EEW_bytes),
      EEW_bytes, data, Store(Data), false, false, false) {
      Ok(true)  => (),
      Ok(false) => internal_error(__FILE__, __LINE__, "store got false from
vmem_write"),
      Err(e)    => return e,
    }
  }
};

set_vstart(zeros());
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvvmmtls (Transposing load/store instructions for matrix tiles)	0.1	Draft

36.11.14. vqwmmacc.vv

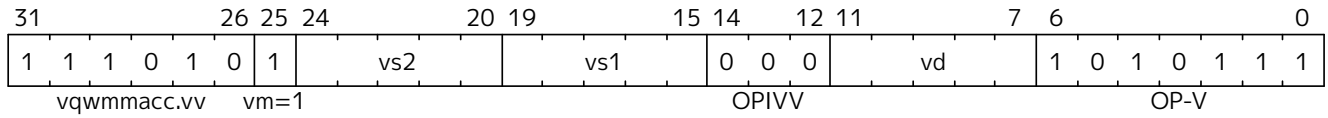
Synopsis

Integer Quad-Widening Matrix Multiply-Accumulate

Mnemonic

vqwmmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Computes the matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$, with a widening factor of 4 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). *vs1* and *vs2* have elements of width $SEW \div 4$; *vd* has elements of width SEW .

$K_{\text{effective}} = \lambda \times 4 \times LMUL$. VL selects how many columns of B (and C) are updated; VL must be a multiple of λ . The signedness of each input operand is selected by `altfmt_A` and `altfmt_B` in `vtype`: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}).

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- VL is not a multiple of λ .
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .

Operation

```

if vm == 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(4, true);

let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);
    let c_bits : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var acc : int = signed(c_bits);
    acc = acc + int_block_dot(i, j, 0, g.K_eff - 1, g, signed_A, signed_B, vs1,
vs2);
    write_single_element(g.EEW_C, c_flat, vd, to_bits_unsafe(g.EEW_C, acc))
  }
}

```

```
};  
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.11.15. vwmacc.vv

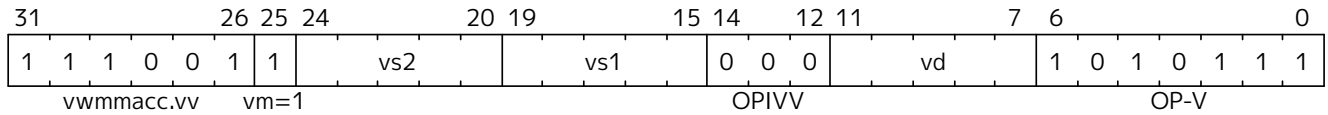
Synopsis

Integer Widening Matrix Multiply-Accumulate

Mnemonic

vwmacc.vv *vd*, *vs1*, *vs2*

Encoding



Description

Computes the matrix-matrix product $T = vs1 \times vs2$ and accumulates it into *vd*: $C \leftarrow C + T$, with a widening factor of 2 applied to the K dimension.

vs1 is interpreted as an $M \times K$ tile (row-major register layout), *vs2* as a $K \times N$ tile (column-major register layout), and *vd* as an $M \times N$ accumulator tile (column-major register layout). *vs1* and *vs2* have elements of width $SEW \div 2$; *vd* has elements of width SEW.

$K_{\text{effective}} = \lambda \times 2 \times \text{LMUL}$. VL selects how many columns of B (and C) are updated; VL must be a multiple of λ . The signedness of each input operand is selected by `altfmt_A` and `altfmt_B` in `vtype`: 0 = signed, 1 = unsigned. Accumulation is modular (wraps at 2^{SEW}).

Exceptions

Illegal Instruction is raised if any of the following conditions holds:

- VL is not a multiple of λ .
- $MUL_C (= VLEN \div (SEW \times \lambda^2))$ is not in $\{1, 2, 4, 8, 16\}$.
- *vs1* or *vs2* is not aligned for LMUL.
- *vd* is not aligned for MUL_C .

Operation

```

if vm == 0 then return Illegal_Instruction();

let g : gemm_geom = decode_gemm_geometry(2, true);

let signed_A : bool = vtype[altfmt_A] == 0b0;
let signed_B : bool = vtype[altfmt_B] == 0b0;

foreach (j from 0 to (g.N - 1)) {
  foreach (i from 0 to (g.M - 1)) {
    let c_flat : int = mat_C_idx(i, j, g.M, g.MUL_C, g.lambda, g.epr_C);
    let c_bits : bits(g.EEW_C) = read_single_element(g.EEW_C, c_flat, vd);
    var acc : int = signed(c_bits);
    acc = acc + int_block_dot(i, j, 0, g.K_eff - 1, g, signed_A, signed_B, vs1,
vs2);
    write_single_element(g.EEW_C, c_flat, vd, to_bits_unsafe(g.EEW_C, acc))
  }
}

```

```
};
RETIRE_SUCCESS
```

Included in

Extension	Minimum version	Lifecycle state
Zvmm (Matrix-multiplication instructions (integer))	0.1	Draft

36.12. Microscaling Subextensions

Each microscaling subextension enables the `vm=0` / `v0.scale` encoding for a specific (input format, accumulator format, block size) combination. A floating-point microscaling subextension implies its corresponding base subextension: for example, `Zvvxofp4fp16mm` implies `Zvvofp4fp16mm`, which provides the underlying OFP4→FP16 instruction support. Only narrow input formats (OFP4, OFP8) define FP microscaling variants. The integer MX subextensions (`Zvvxi*mm`, `Zvvxni*mm`) do not imply a base subextension: `vfwimmacc.vv` and `vfqwimmacc.vv` have no non-MX FP-accumulate form (the `vm=1` encoding of those opcodes is the existing integer-accumulate `vmmacc.vv` / `vqwmacc.vv`). The block size and implication relationships are listed in [Table 83](#).

Table 83. Microscaling subextensions

Extension	BS	Implies	Multiplicand Types	Accumulator Type
<code>Zvvxofp4ofp8mm</code>	32	<code>Zvvofp4ofp8mm</code>	MXFP4 (E8M0-scaled OFP4), MXFP4	OFP8
<code>Zvvxofp4fp16mm</code>	32	<code>Zvvofp4fp16mm</code>	MXFP4, MXFP4	IEEE binary16
<code>Zvvxofp4bf16mm</code>	32	<code>Zvvofp4bf16mm</code>	MXFP4, MXFP4	BFloat16
<code>Zvvxofp8mm</code>	32	<code>Zvvofp8mm</code>	MXFP8 (E8M0-scaled OFP8), MXFP8	OFP8
<code>Zvvxofp8fp16mm</code>	32	<code>Zvvofp8fp16mm</code>	MXFP8, MXFP8	IEEE binary16
<code>Zvvxofp8bf16mm</code>	32	<code>Zvvofp8bf16mm</code>	MXFP8, MXFP8	BFloat16
<code>Zvvxofp8fp32mm</code>	32	<code>Zvvofp8fp32mm</code>	MXFP8, MXFP8	IEEE binary32
<code>Zvvxofp4fp32mm</code>	32	<code>Zvvofp4fp32mm</code>	MXFP4, MXFP4	IEEE binary32
<code>Zvvxofp8fp64mm</code>	32	<code>Zvvofp8fp64mm</code>	MXFP8, MXFP8	IEEE binary64
<code>Zvvxnofp4ofp8mm</code>	16	<code>Zvvxofp4ofp8mm</code>	MXFP4 (E8M0-scaled OFP4), MXFP4	OFP8
<code>Zvvxnofp4fp16mm</code>	16	<code>Zvvxofp4fp16mm</code>	MXFP4, MXFP4	IEEE binary16
<code>Zvvxnofp4bf16mm</code>	16	<code>Zvvxofp4bf16mm</code>	MXFP4, MXFP4	BFloat16
<code>Zvvxnofp8mm</code>	16	<code>Zvvxofp8mm</code>	MXFP8 (E8M0-scaled OFP8), MXFP8	OFP8

Extension	BS	Implies	Multiplicand Types	Accumulator Type
Zvvxnopf8fp16mm	16	Zvvxopf8fp16mm	MXFP8, MXFP8	IEEE binary16
Zvvxnopf8bf16mm	16	Zvvxopf8bf16mm	MXFP8, MXFP8	BFloat16
Zvvxnopf8fp32mm	16	Zvvxopf8fp32mm	MXFP8, MXFP8	IEEE binary32
Zvvxnopf4fp32mm	16	Zvvxopf4fp32mm	MXFP4, MXFP4	IEEE binary32
Zvvxnopf8fp64mm	16	Zvvxopf8fp64mm	MXFP8, MXFP8	IEEE binary64
Zvvxi8fp16mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary16
Zvvxi8bf16mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	BFloat16
Zvvxi8fp32mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary32
Zvvxi8fp64mm	32	—	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary64
Zvvxi4fp16mm	32	—	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary16
Zvvxi4bf16mm	32	—	MXINT4 (E8M0-scaled Int4), MXINT4	BFloat16
Zvvxi4fp32mm	32	—	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary32
Zvvxni8fp16mm	16	Zvvxi8fp16mm	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary16
Zvvxni8bf16mm	16	Zvvxi8bf16mm	MXINT8 (E8M0-scaled Int8), MXINT8	BFloat16
Zvvxni8fp32mm	16	Zvvxi8fp32mm	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary32
Zvvxni8fp64mm	16	Zvvxi8fp64mm	MXINT8 (E8M0-scaled Int8), MXINT8	IEEE binary64
Zvvxni4fp16mm	16	Zvvxi4fp16mm	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary16
Zvvxni4bf16mm	16	Zvvxi4bf16mm	MXINT4 (E8M0-scaled Int4), MXINT4	BFloat16
Zvvxni4fp32mm	16	Zvvxi4fp32mm	MXINT4 (E8M0-scaled Int4), MXINT4	IEEE binary32



See [Section 36.7.3.3](#) for LMUL restrictions when **bs=1**.

36.13. Encoding map

The following tables enumerate every (instruction, SEW, altfmt) encoding point and map it to its input and output data types and the governing subextension. Entries marked *reserved* raise an illegal-instruction exception.

36.13.1. Floating-point encoding map

Mixed-format inputs (`altfmt_A` $\neq$ `altfmt_B`) are permitted wherever both input formats are defined for a given (SEW, W) combination; those combinations are listed explicitly in the table below.

Table 84. Floating-point multiply-accumulate encoding map

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Subextension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vfmacc.vv (W=1)											
vfmacc	8	8	0	E4M3	0	E4M3	0	E4M3	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	8	8	0	E4M3	0	E4M3	1	E5M2	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	8	8	1	E5M2	1	E5M2	0	E4M3	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	8	8	1	E5M2	1	E5M2	1	E5M2	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	8	8	0	E4M3	1	E5M2	0	E4M3	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	8	8	0	E4M3	1	E5M2	1	E5M2	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	8	8	1	E5M2	0	E4M3	0	E4M3	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	8	8	1	E5M2	0	E4M3	1	E5M2	Zvvofp8mm	Zvvxofp8mm	Zvvxnofp8mm
vfmacc	16	16	0	FP16	0	FP16	0	FP16	Zvvfp16mm	—	—
vfmacc	16	16	0	FP16	0	FP16	1	BF16	<i>reserved</i>	—	—
vfmacc	16	16	1	BF16	1	BF16	0	FP16	<i>reserved</i>	—	—
vfmacc	16	16	1	BF16	1	BF16	1	BF16	Zvvbf16mm	—	—
vfmacc	16	16	0	FP16	1	BF16	0	FP16	Zvvfp16mm, Zvvbf16mm	—	—
vfmacc	16	16	0	FP16	1	BF16	1	BF16	Zvvfp16mm, Zvvbf16mm	—	—
vfmacc	16	16	1	BF16	0	FP16	0	FP16	Zvvfp16mm, Zvvbf16mm	—	—
vfmacc	16	16	1	BF16	0	FP16	1	BF16	Zvvfp16mm, Zvvbf16mm	—	—
vfmacc	32	32	×	FP32	×	FP32	0	FP32	Zvvfp32mm	—	—

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Subextension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vfmacc	32	32	×	FP32	×	FP32	1	—	<i>reserved</i>	—	—
vfmacc	64	64	×	FP64	×	FP64	0	FP64	Zvvp64mm	—	—
vfmacc	64	64	×	FP64	×	FP64	1	—	<i>reserved</i>	—	—
vfwmmacc.vv (W=2)											
vfwmmacc	8	4	0	E2M1	0	E2M1	0	E4M3	Zvvofp4ofp8mm	Zvvxofp4ofp8mm	Zvvxnopf4ofp8mm
vfwmmacc	8	4	0	E2M1	0	E2M1	1	E5M2	Zvvofp4ofp8mm	Zvvxofp4ofp8mm	Zvvxnopf4ofp8mm
vfwmmacc	8	4	1	—	1	—	×	—	<i>reserved</i>	—	—
vfwmmacc	16	8	0	E4M3	0	E4M3	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmmacc	16	8	0	E4M3	0	E4M3	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmmacc	16	8	1	E5M2	1	E5M2	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmmacc	16	8	1	E5M2	1	E5M2	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmmacc	16	8	0	E4M3	1	E5M2	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmmacc	16	8	0	E4M3	1	E5M2	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmmacc	16	8	1	E5M2	0	E4M3	0	FP16	Zvvofp8fp16mm	Zvvxofp8fp16mm	Zvvxnopf8fp16mm
vfwmmacc	16	8	1	E5M2	0	E4M3	1	BF16	Zvvofp8bfp16mm	Zvvxofp8bfp16mm	Zvvxnopf8bfp16mm
vfwmmacc	32	16	0	FP16	0	FP16	0	FP32	Zvvp16fp32mm	—	—
vfwmmacc	32	16	0	FP16	0	FP16	1	—	<i>reserved</i>	—	—
vfwmmacc	32	16	1	BF16	1	BF16	0	FP32	Zvvp16fp32mm	—	—
vfwmmacc	32	16	1	BF16	1	BF16	1	—	<i>reserved</i>	—	—
vfwmmacc	32	16	0	FP16	1	BF16	0	FP32	Zvvp16fp32mm, Zvvp16fp32mm	—	—
vfwmmacc	32	16	1	BF16	0	FP16	0	FP32	Zvvp16fp32mm, Zvvp16fp32mm	—	—
vfwmmacc	64	32	×	FP32	×	FP32	0	FP64	Zvvp32fp64mm	—	—
vfwmmacc	64	32	×	FP32	×	FP32	1	—	<i>reserved</i>	—	—
vfqmmacc.vv (W=4)											

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Subextension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vfqwmmacc	8	2	×	—	×	—	×	—	<i>reserved</i>	—	—
vfqwmmacc	16	4	0	E2M1	0	E2M1	0	FP16	Zvvofp4fp16mm	Zvvxofp4fp16mm	Zvvxnofp4fp16mm
vfqwmmacc	16	4	0	E2M1	0	E2M1	1	BF16	Zvvofp4bf16mm	Zvvxofp4bf16mm	Zvvxnofp4bf16mm
vfqwmmacc	16	4	1	—	1	—	×	—	<i>reserved</i>	—	—
vfqwmmacc	32	8	0	E4M3	0	E4M3	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqwmmacc	32	8	0	E4M3	0	E4M3	1	—	<i>reserved</i>	—	—
vfqwmmacc	32	8	1	E5M2	1	E5M2	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqwmmacc	32	8	1	E5M2	1	E5M2	1	—	<i>reserved</i>	—	—
vfqwmmacc	32	8	0	E4M3	1	E5M2	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqwmmacc	32	8	1	E5M2	0	E4M3	0	FP32	Zvvofp8fp32mm	Zvvxofp8fp32mm	Zvvxnofp8fp32mm
vfqwmmacc	64	16	0	FP16	0	FP16	0	FP64	Zvvfp16fp64mm	—	—
vfqwmmacc	64	16	0	FP16	0	FP16	1	—	<i>reserved</i>	—	—
vfqwmmacc	64	16	1	BF16	1	BF16	0	FP64	Zvvbf16fp64mm	—	—
vfqwmmacc	64	16	1	BF16	1	BF16	1	—	<i>reserved</i>	—	—
vfqwmmacc	64	16	0	FP16	1	BF16	0	FP64	Zvvfp16fp64mm, Zvvbf16fp64mm	—	—
vfqwmmacc	64	16	1	BF16	0	FP16	0	FP64	Zvvfp16fp64mm, Zvvbf16fp64mm	—	—
vf8wmmacc.vv (W=8)											
vf8wmmacc	8	1	×	—	×	—	×	—	<i>reserved</i>	—	—
vf8wmmacc	16	2	×	—	×	—	×	—	<i>reserved</i>	—	—
vf8wmmacc	32	4	0	E2M1	0	E2M1	0	FP32	Zvvofp4fp32mm	Zvvxofp4fp32mm	Zvvxnofp4fp32mm
vf8wmmacc	32	4	0	E2M1	0	E2M1	1	—	<i>reserved</i>	—	—

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	Subextension	MX BS=32 (vm=0)	MX BS=16 (vm=0)
vf8wmmacc	32	4	1	—	1	—	×	—	<i>reserved</i>	—	—
vf8wmmacc	64	8	0	E4M3	0	E4M3	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm
vf8wmmacc	64	8	0	E4M3	0	E4M3	1	—	<i>reserved</i>	—	—
vf8wmmacc	64	8	1	E5M2	1	E5M2	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm
vf8wmmacc	64	8	1	E5M2	1	E5M2	1	—	<i>reserved</i>	—	—
vf8wmmacc	64	8	0	E4M3	1	E5M2	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm
vf8wmmacc	64	8	1	E5M2	0	E4M3	0	FP64	Zvvofp8fp64mm	Zvvxofp8fp64mm	Zvvxnofp8fp64mm

36.13.2. Integer encoding map

altfmt_A and **altfmt_B** select signed (0) or unsigned (1) interpretation; the accumulator is always signed. **altfmt** is unused and **vm** must be 1 (**vm=0** is *reserved*) for all four instructions. For **vmmacc.vv**, **vqwmacc.vv**, and **v8wmacc.vv**, **vm=0** is separately defined as **vwimmacc.vv**, **vfqwmacc.vv**, and **vf8wmacc.vv** (integer inputs, FP accumulator, E8M0 microscaling); see [Section 36.13.3](#).

Table 85. Integer multiply-accumulate encoding map

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	C Type	Subextension
vmmacc.vv (W=1)								
vmmacc	8	8	0	Int8	0	Int8	Int8	Zvvi8mm
vmmacc	8	8	0	Int8	1	UInt8	Int8	Zvvi8mm
vmmacc	8	8	1	UInt8	0	Int8	Int8	Zvvi8mm
vmmacc	8	8	1	UInt8	1	UInt8	Int8	Zvvi8mm
vmmacc	16	16	0	Int16	0	Int16	Int16	Zvvi16mm
vmmacc	16	16	0	Int16	1	UInt16	Int16	Zvvi16mm
vmmacc	16	16	1	UInt16	0	Int16	Int16	Zvvi16mm
vmmacc	16	16	1	UInt16	1	UInt16	Int16	Zvvi16mm
vmmacc	32	32	0	Int32	0	Int32	Int32	Zvvi32mm
vmmacc	32	32	0	Int32	1	UInt32	Int32	Zvvi32mm
vmmacc	32	32	1	UInt32	0	Int32	Int32	Zvvi32mm
vmmacc	32	32	1	UInt32	1	UInt32	Int32	Zvvi32mm
vmmacc	64	64	0	Int64	0	Int64	Int64	Zvvi64mm
vmmacc	64	64	0	Int64	1	UInt64	Int64	Zvvi64mm
vmmacc	64	64	1	UInt64	0	Int64	Int64	Zvvi64mm
vmmacc	64	64	1	UInt64	1	UInt64	Int64	Zvvi64mm

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	C Type	Subextension
vwmacc.vv (W=2)								
vwmacc	8	4	0	Int4	0	Int4	Int8	Zvvi4i8mm
vwmacc	8	4	0	Int4	1	UInt4	Int8	Zvvi4i8mm
vwmacc	8	4	1	UInt4	0	Int4	Int8	Zvvi4i8mm
vwmacc	8	4	1	UInt4	1	UInt4	Int8	Zvvi4i8mm
vwmacc	16	8	0	Int8	0	Int8	Int16	Zvvi8i16mm
vwmacc	16	8	0	Int8	1	UInt8	Int16	Zvvi8i16mm
vwmacc	16	8	1	UInt8	0	Int8	Int16	Zvvi8i16mm
vwmacc	16	8	1	UInt8	1	UInt8	Int16	Zvvi8i16mm
vwmacc	32	16	0	Int16	0	Int16	Int32	Zvvi16i32mm
vwmacc	32	16	0	Int16	1	UInt16	Int32	Zvvi16i32mm
vwmacc	32	16	1	UInt16	0	Int16	Int32	Zvvi16i32mm
vwmacc	32	16	1	UInt16	1	UInt16	Int32	Zvvi16i32mm
vwmacc	64	32	0	Int32	0	Int32	Int64	Zvvi32i64mm
vwmacc	64	32	0	Int32	1	UInt32	Int64	Zvvi32i64mm
vwmacc	64	32	1	UInt32	0	Int32	Int64	Zvvi32i64mm
vwmacc	64	32	1	UInt32	1	UInt32	Int64	Zvvi32i64mm
vqwmacc.vv (W=4)								
vqwmacc	8	2	×	—	×	—	—	<i>reserved</i>
vqwmacc	16	4	0	Int4	0	Int4	Int16	Zvvi4i16mm
vqwmacc	16	4	0	Int4	1	UInt4	Int16	Zvvi4i16mm
vqwmacc	16	4	1	UInt4	0	Int4	Int16	Zvvi4i16mm
vqwmacc	16	4	1	UInt4	1	UInt4	Int16	Zvvi4i16mm
vqwmacc	32	8	0	Int8	0	Int8	Int32	Zvvi8i32mm
vqwmacc	32	8	0	Int8	1	UInt8	Int32	Zvvi8i32mm
vqwmacc	32	8	1	UInt8	0	Int8	Int32	Zvvi8i32mm
vqwmacc	32	8	1	UInt8	1	UInt8	Int32	Zvvi8i32mm
vqwmacc	64	16	0	Int16	0	Int16	Int64	Zvvi16i64mm
vqwmacc	64	16	0	Int16	1	UInt16	Int64	Zvvi16i64mm
vqwmacc	64	16	1	UInt16	0	Int16	Int64	Zvvi16i64mm
vqwmacc	64	16	1	UInt16	1	UInt16	Int64	Zvvi16i64mm
v8wmacc.vv (W=8)								
v8wmacc	8	1	×	—	×	—	—	<i>reserved</i>
v8wmacc	16	2	×	—	×	—	—	<i>reserved</i>
v8wmacc	32	4	0	Int4	0	Int4	Int32	Zvvi4i32mm
v8wmacc	32	4	0	Int4	1	UInt4	Int32	Zvvi4i32mm
v8wmacc	32	4	1	UInt4	0	Int4	Int32	Zvvi4i32mm
v8wmacc	32	4	1	UInt4	1	UInt4	Int32	Zvvi4i32mm
v8wmacc	64	8	0	Int8	0	Int8	Int64	Zvvi8i64mm

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	C Type	Subextension
v8wmmacc	64	8	0	Int8	1	UInt8	Int64	Zvvi8i64mm
v8wmmacc	64	8	1	UInt8	0	Int8	Int64	Zvvi8i64mm
v8wmmacc	64	8	1	UInt8	1	UInt8	Int64	Zvvi8i64mm

36.13.3. Integer MX encoding map (vm=0)

When `vm=0`, `vwmacc.vv`, `vqwmacc.vv`, and `v8wmmacc.vv` are decoded as `vwimmacc.vv`, `vfqwimmacc.vv`, and `vf8wimmacc.vv` respectively. MXINT inputs are always signed; `altfmt_A` and `altfmt_B` must be 0 (unsigned is *reserved*). `altfmt` selects the FP accumulator format. `vwmacc.vv` with `vm=0` remains *reserved*.

Table 86. Integer MX multiply-accumulate encoding map (vm=0)

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	MX BS=32	MX BS=16
vwimmacc.vv (W=2, vm=0 of vwmacc opcode)										
vwimmacc	8	4	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vwimmacc	16	8	0	Int8	0	Int8	0	FP16	Zvvxi8fp16mm	Zvvxni8fp16mm
vwimmacc	16	8	0	Int8	1	UInt8	0	FP16	<i>reserved</i>	<i>reserved</i>
vwimmacc	16	8	1	UInt8	0	Int8	0	FP16	<i>reserved</i>	<i>reserved</i>
vwimmacc	16	8	1	UInt8	1	UInt8	0	FP16	<i>reserved</i>	<i>reserved</i>
vwimmacc	16	8	0	Int8	0	Int8	1	BF16	Zvvxi8bf16mm	Zvvxni8bf16mm
vwimmacc	16	8	0	Int8	1	UInt8	1	BF16	<i>reserved</i>	<i>reserved</i>
vwimmacc	16	8	1	UInt8	0	Int8	1	BF16	<i>reserved</i>	<i>reserved</i>
vwimmacc	16	8	1	UInt8	1	UInt8	1	BF16	<i>reserved</i>	<i>reserved</i>
vwimmacc	32	16	×	Int16	×	Int16	×	—	<i>reserved</i>	<i>reserved</i>
vwimmacc	64	32	×	Int32	×	Int32	×	—	<i>reserved</i>	<i>reserved</i>
vfqwimmacc.vv (W=4, vm=0 of vqwmacc opcode)										
vfqwimmacc	8	2	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vfqwimmacc	16	4	0	Int4	0	Int4	0	FP16	Zvvxi4fp16mm	Zvvxni4fp16mm
vfqwimmacc	16	4	0	Int4	1	UInt4	0	FP16	<i>reserved</i>	<i>reserved</i>
vfqwimmacc	16	4	1	UInt4	0	Int4	0	FP16	<i>reserved</i>	<i>reserved</i>
vfqwimmacc	16	4	1	UInt4	1	UInt4	0	FP16	<i>reserved</i>	<i>reserved</i>
vfqwimmacc	16	4	0	Int4	0	Int4	1	BF16	Zvvxi4bf16mm	Zvvxni4bf16mm
vfqwimmacc	16	4	0	Int4	1	UInt4	1	BF16	<i>reserved</i>	<i>reserved</i>
vfqwimmacc	16	4	1	UInt4	0	Int4	1	BF16	<i>reserved</i>	<i>reserved</i>

Instruction	SEW	EEW	altfmt_A	Input A	altfmt_B	Input B	altfmt	C Type	MX BS=32	MX BS=16
vfqwimmac	16	4	1	UInt4	1	UInt4	1	BF16	<i>reserved</i>	<i>reserved</i>
vfqwimmac	32	8	0	Int8	0	Int8	0	FP32	Zvvxi8fp32mm	Zvvxni8fp32mm
vfqwimmac	32	8	0	Int8	1	UInt8	0	FP32	<i>reserved</i>	<i>reserved</i>
vfqwimmac	32	8	1	UInt8	0	Int8	0	FP32	<i>reserved</i>	<i>reserved</i>
vfqwimmac	32	8	1	UInt8	1	UInt8	0	FP32	<i>reserved</i>	<i>reserved</i>
vfqwimmac	32	8	×	—	×	—	1	—	<i>reserved</i>	<i>reserved</i>
vfqwimmac	64	16	×	Int16	×	Int16	×	—	<i>reserved</i>	<i>reserved</i>
vf8wimmacc.vv (W=8, vm=0 of v8wmmacc opcode)										
vf8wimmac	8	1	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vf8wimmac	16	2	×	—	×	—	×	—	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	0	Int4	0	Int4	0	FP32	Zvvxi4fp32mm	Zvvxni4fp32mm
vf8wimmac	32	4	0	Int4	1	UInt4	0	FP32	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	1	UInt4	0	Int4	0	FP32	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	1	UInt4	1	UInt4	0	FP32	<i>reserved</i>	<i>reserved</i>
vf8wimmac	32	4	×	—	×	—	1	—	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	0	Int8	0	Int8	0	FP64	Zvvxi8fp64mm	Zvvxni8fp64mm
vf8wimmac	64	8	0	Int8	1	UInt8	0	FP64	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	1	UInt8	0	Int8	0	FP64	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	1	UInt8	1	UInt8	0	FP64	<i>reserved</i>	<i>reserved</i>
vf8wimmac	64	8	×	—	×	—	1	—	<i>reserved</i>	<i>reserved</i>

[1] Open Compute Project, *OCF Microscaling Formats (MX) v1.0 Spec*, www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf

[2] S = R: fully packed, no padding